

A Major Project Report On

**LOW POWER FAULT TOLERANT REVERSIBLE
DECODER USING MOS TRANSISTORS**

Submitted to

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
Hyderabad (A.P.)**

In partial fulfillment of the requirement for the award of Degree of

BACHELOR OF TECHNOLOGY

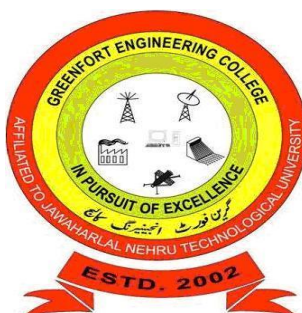
IN

ELECTRONICS & COMMUNICATION ENGINEERING

SUBMITTED BY

MEHMOOD ALI

12J01A0440



DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

GREENFORT ENGINEERING COLLEGE

Bandlaguda, Near Chandrayangutta X Road Keshavgiri (P),

Hyderabad-05, TS, India



**GREENFORT ENGINEERING
COLLEGE** Bandlaguda, Near Chandrayangutta X Road
Keshavgiri (P),
Hyderabad-05, TS, India

CERTIFICATE

This is to certify that the project entitled

LOW POWER FAULT TOLERANT REVERSIBLE DECODER USING MOS TRANSISTORS

Submitted in the partial fulfillment of the requirement for the Degree of
BACHELOR OF TECHNOLOGY

Of the

Jawaharlal Nehru Technological University Hyderabad, Andhra Pradesh

Is record of bonafide work carried out by

MEHMOOD ALI

12J01A0440

Under our supervision and that no part of this project has been submitted for the award of any other Degree, Diploma, Fellowship or similar titles or prizes and that the work has not been published in scientific or popular journal or magazine.

Mr. RASHED NAWAZ KHAN

Mrs AZEEZ KHATOON

INTERNAL GUIDE

HEAD OF THE DEPARTMENT

EXTERNAL

Mr. KHAJA MOHINUDDIN

PRINCIPAL

DECLARATION

This is to declare that the project entitled
**LOW POWER FAULT TOLERANT REVERSIBLE DECODER USING MOS
TRANSISTORS** submitted by me in partial fulfillment of the
requirement for the award of the degree of Bachelor of Technology
in **ELECTRONICS & COMMUNICATION ENGINEERING** during the
period Jan-march 2015.

SUBMITTED BY :

MEHMOOD ALI

12J01A0440

ACKNOWLEDGEMENT

I extend my warm respect to our Principal **Mr. KHAJA MOHINUDDIN** for his moral support and for providing infrastructure, computer and lab facilities.

I wish to acknowledge our sincere thanks to our Head of the Department (ECE), **Mrs AZEEZ KHATOON** for her permission to carry out the project work at Innovative Technologies ,Hyderabad.

I thankful to all faculty members of ECE Department and the non-teaching staff, Librarians and all those who have directly or indirectly helped us in the completion of the Project.

I express my wholehearted indebtedness and deep sense of gratitude to who has lot of experience in industries for his guidance, help, counseling and constant encouragement throughout the Project.

With sincere regards,

MEHMOOD ALI
12J01A0440

Table of Contents

1.1 VLSI:	8
1.1.1 : HISTORY:	8
1.1.2: DEVELOPMENT:	9
1.1.2: STRUCTURED DESIGN:	10
1.2: VLSI DESIGN METHODOLOGY	5
1.3: FRONT END OF IC FABRICATION	6
1.4: BACK END OF IC FABRICATION:	6
1.5: Floor Planning:	8
1.5.1: The floorplan-based design methodology	8
Core Boundary	9
IO Placement/Pin placement	9
Macro placement	10
4. Creating Power Rings and Straps	10
1.5.2 Floorplanning concepts	11
1.5.2.1 Abutment: Establishing connections between cells by putting them directly next to each other, without the necessity of routing as shown in figure 3.12.2.1	11
1.5.2.2 Leaf cell:	11
1.5.2.3 Composite cell:	11
1.5.2.4 Slicing floorplans:	11
1.6: PLACEMENT:	12
1.6.2: Important things in placement:	12
1. High fanout net synthesis	12
2. Use Ideal clock	12
Control Congestion	13
Macro-padding:	13
5. Maximum Utilization constraint:	13
Placement blockages:	13
Scan chain reordering:	13
TIE cells	13
1.7: Clock Tree Synthesis	14
1.7.3: Setup and Hold Fixing	16
Set Up Fixing:	16
Hold Fixing:	16
1.8: ROUTING	17
1.4.1: Routing Congestion	18

1.4.2: Routing Order.....	18
1.4.2.1: Power Routing:.....	19
1.4.2.2: Clock Routing:.....	19
1.4.2.3: Signal Routing:.....	19
1.9: Signal Integrity.....	19
2.1: The MOSFET as A Switch:	22
2.2: MOSFET Characteristics Curves.....	22
2.3: DIGITAL-CIRCUIT SPEED AND POWER CONSUMPTION.....	23
2.4: MOS LOGIC.....	25
2.4.1: Signal Strengths.....	25
2.5: COMBINATIONAL & SEQUENTIAL LOGIC.....	26
2.5.1 combinational logic representation:.....	27
2.5.2: Sequential Logic Representation.....	29
2.5.3: Classification of Sequential Logic.....	29
2.3.4: Synchronous sequential circuit	30
4.6: SYNCHRONIZATION.....	30
2.3.2: Process of synchronization.....	31
1) Planning.....	31
2) Ownership.....	31
3) Scheduling.....	31
4) Monitoring.....	31
2.3.2.1: Challenges of synchronization.....	31
1) Data Formats Complexity.....	31
2) Real-timeliness.....	32
3) Security.....	32
4) Data Quality.....	32
5) Performance.....	32
6) Maintenance.....	33
2.6.3: Setup & Hold Time	33
2.6.4: Metastability	35
Causes of metastability:.....	37
4.7: ADVANCED LOW POWER TECHNIQUES.....	38
2.3.3: Techniques to reduce dynamic and leakage power.....	38
3.1.1 REVERSIBLE AND FAULT TOLERANT GATES:.....	41
3.1.2. QUBIT AND QUANTUM COST.....	41
3.1.3. DELAY, GARBAGE OUTPUT AND HARDWARE COMPLEXITY:.....	42

3.1.4. POPULAR REVERSIBLE FAULT TOLERANT GATES.....	43
3.1.4.1: FREDKIN GATE:	43
3.1.4.2: TOFFOLI GATES:	43
3.1.4.3: PERES GATE:.....	44
3.1.4.4: FEYNMAN DOUBLE GATE:.....	45
3.1.5: DECODER:.....	46
3.2 BLOCK DIAGRAM OF FEYNMAN DOUBLE GATE.....	47
3.2.1 SCHEMATIC AND LAYOUT OF FEYNMAN DOUBLE GATE.....	49
3.3.1 GRAPH AND TIMING DIAGRAM.....	50
3.3: BLOCK DIAGRAM OF FREDKIN GATE:	51
3.3.2 SCHEMATIC AND LAYOUT OF FEDKIN GATE.....	53
3.3.3 GRAPH AND TIMING DIAGRAM OF FREDKIN GATE.....	54
4.1 BLOCK DIAGRAM OF PROPOSED FAULT.....	55
TOLERANT REVERSIBLE DECODER.....	55
4.2: SCHEMATIC & LAYOUT OF PROPOSED DECODER.....	57
4.3 GRAPH AND TIMING DIAGRAM OF PROPOSED DECODER.....	58
4.4: ALGORITHM OF THE PROPOSED DECODER.....	59
5.1 ABOUT MICROWIND.....	60
5.2 ABOUT DSCH.....	61
5.2.1 HIGHLIGHTS.....	61
5.3 SOFTWARE STEPS.....	62
6.1 ADVANTAGES:	68
6.2 DISADVANTAGES:	68
6.3 APPLICATIONS.....	68
7.1 CONCLUSION.....	69
8.1 BIBLIOGRAPHY.....	70

1.1 VLSI:

Very-large-scale integration (VLSI) is the process of creating an integrated circuit (IC) by combining thousands of transistors into a single chip. VLSI began in the 1970s when complex semiconductor and communication technologies were being developed. The microprocessor is a VLSI device. Before the introduction of VLSI technology most ICs had a limited set of functions they could perform. An electronic circuit might consist of a CPU, ROM, RAM and other glue logic. VLSI lets IC designers add all of these into one chip

1.1.1 : HISTORY:

During the mid-1920s, several inventors attempted devices that were intended to control current in solid-state diodes and convert them into triodes. Success did not come until after WWII, during which the attempt to improve silicon and germanium crystals for use as radar detectors led to improvements in fabrication and in the understanding of

quantum mechanical states of carriers in semiconductors. Then scientists who had been diverted to radar development returned to solid-state device development. With the invention of transistors at Bell Labs in 1947, the field of electronics shifted from vacuum tubes to solid-state devices. With the small transistor at their hands, electrical engineers of the 1950s saw the possibilities of constructing far more advanced circuits. As the complexity of circuits grew, problems arose.

One problem was the size of the circuit. A complex circuit, like a computer, was dependent on speed. If the components of the computer were too large or the wires interconnecting them too long, the electric signals couldn't travel fast enough through the circuit, thus making the computer too slow to be effective

Jack Kilby at Texas Instruments found a solution to this problem in 1958. Kilby's idea was to make all the components and the chip out of the same block (monolith) of semiconductor material. Kilby presented his idea to his superiors, and was allowed to build a test version of his circuit. In September 1958, he had his first integrated circuit ready.^[1] Although the first integrated circuit was crude and had some problems, the idea was groundbreaking. By making all the parts out of the same block of material and adding the metal needed to connect them as a layer on top of it, there was no need for discrete components. No more wires and components had to be assembled manually. The circuits could be made smaller, and the manufacturing process could be automated. From here, the idea of integrating all components on a single silicon wafer came into existence, which led to development in small-scale integration (SSI) in the early 1960s, medium-scale integration (MSI) in the late 1960s, and then large-scale integration (LSI) as well as VLSI in the 1970s and 1980s, with tens of thousands of transistors on a single chip (later hundreds of thousands, then millions, and now billions (10^9)).

1.1.2: DEVELOPMENT:

The first semiconductor chips held two transistors each. Subsequent advances added more transistors, and as a consequence, more individual functions or systems were integrated over time. The first integrated circuits held only a few devices, perhaps as

many as ten diodes, transistors, resistors and capacitors, making it possible to fabricate one or more logic gates on a single device. Now known retrospectively as *small-scale integration* (SSI), improvements in technique led to devices with hundreds of logic gates, known as *medium-scale integration* (MSI). Further improvements led to *large-scale integration* (LSI), i.e. systems with at least a thousand logic gates. Current technology has moved far past this mark and today's microprocessors have many millions of gates and billions of individual transistors.

At one time, there was an effort to name and calibrate various levels of large-scale integration above VLSI. Terms like *ultra-large-scale integration* (ULSI) were used. But the huge number of gates and transistors available on common devices has rendered such fine distinctions moot. Terms suggesting greater than VLSI levels of integration are no longer in widespread use.

As of early 2008, billion-transistor processors are commercially available. This became more commonplace as semiconductor fabrication advanced from the then-current generation of 65 nm processes. Current designs, unlike the earliest devices, use extensive design automation and automated logic synthesis to lay out the transistors, enabling higher levels of complexity in the resulting logic functionality. Certain high-performance logic blocks like the SRAM (static random-access memory) cell, are still designed by hand to ensure the highest efficiency. VLSI technology may be moving toward further radical miniaturization with introduction of NEMS technology.

1.1.2: STRUCTURED DESIGN:

Structured VLSI design is a modular methodology originated by Carver Mead and Lynn Conway for saving microchip area by minimizing the interconnect fabrics area. This is obtained by repetitive arrangement of rectangular macro blocks which can be interconnected using wiring by abutment.

An example is partitioning the layout of an adder into a row of equal bit slices cells. In complex designs this structuring may be achieved by hierarchical nesting. Structured VLSI design had been popular in the early 1980s, but lost its popularity later because of the advent of placement and routing tools wasting a lot of area by routing, which is tolerated because of the progress of Moore's Law.

When introducing the hardware description language KARL in the mid' 1970s, Reiner Hartenstein coined the term "structured VLSI design" (originally as "structured LSI design"), echoing Edsger Dijkstra's structured programming approach by procedure nesting to avoid chaotic *spaghetti-structured* program

1.2: VLSI DESIGN METHODOLOGY

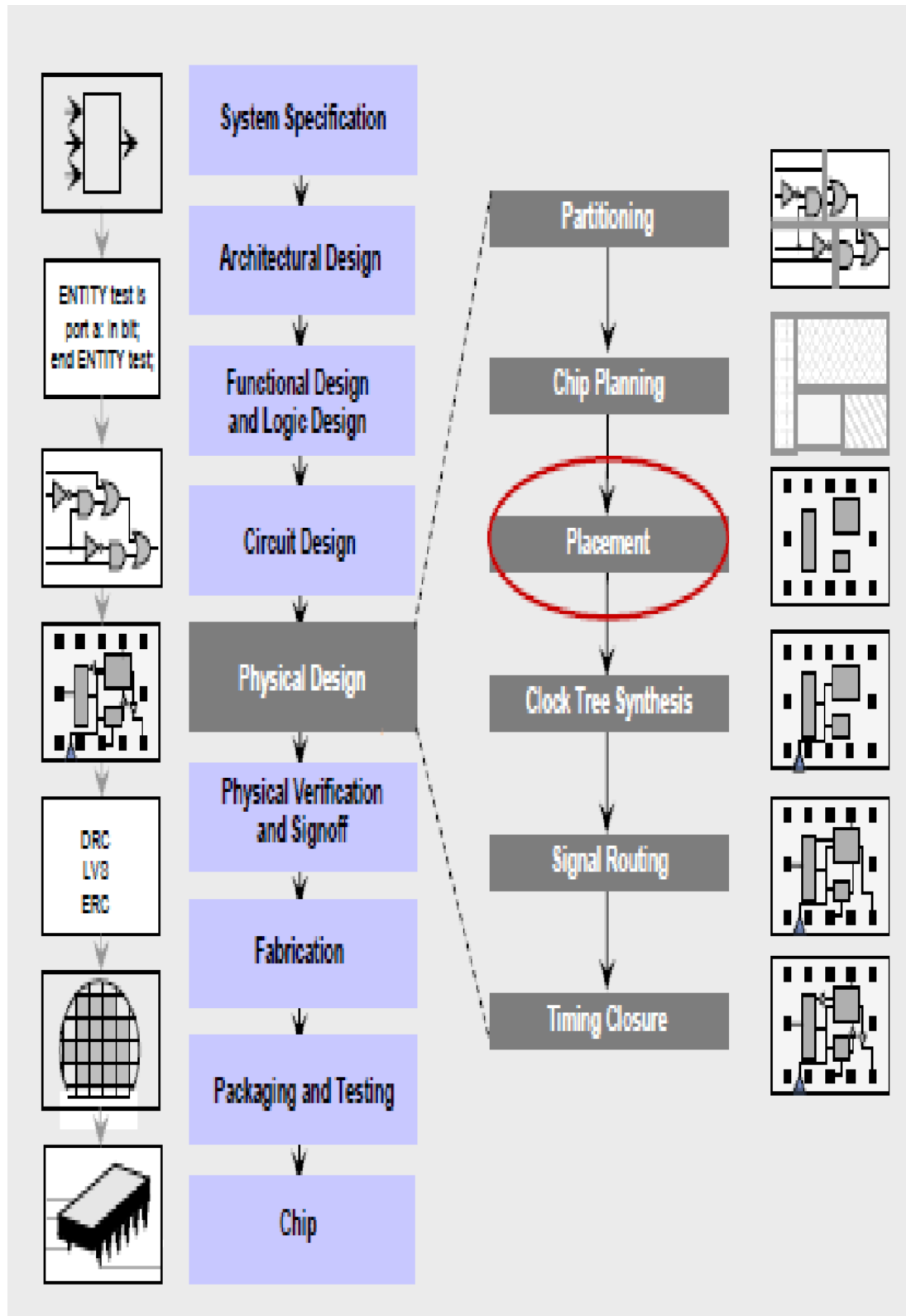


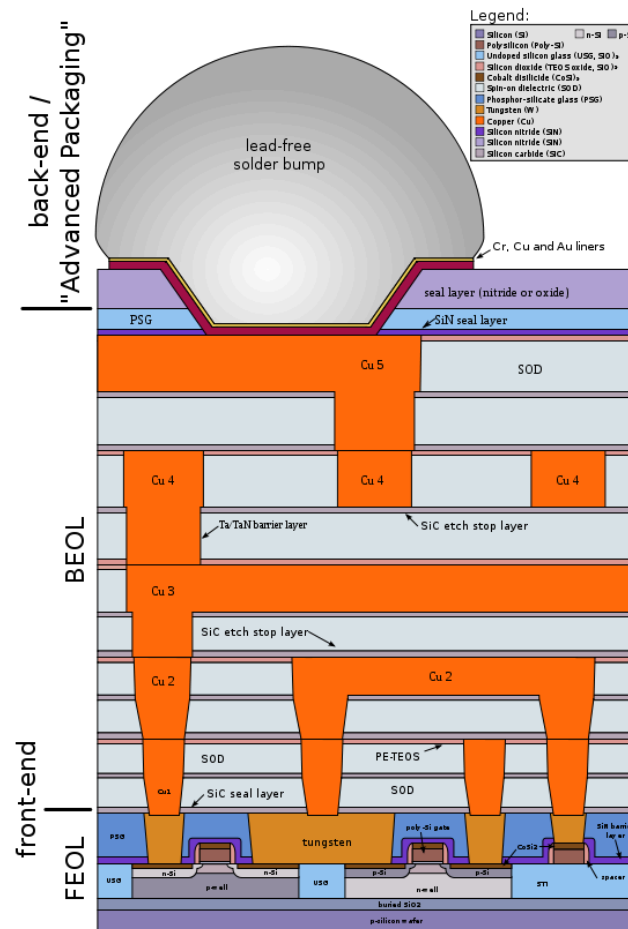
Figure 1.2.1 Flowchart of design methodology

1.3: FRONT END OF IC FABRICATION

The front-end (FE) is the first portion of IC fabrication where the individual devices (transistors, capacitors, resistors, etc.) are patterned in the semiconductor FE generally covers everything up to (but not including) the deposition of metal interconnect layers.

FE contains all processes of CMOS fabrication needed to form fully isolated CMOS elements:

1. Selecting the type of wafer to be used; Chemical-mechanical planarization and cleaning of the wafer.
2. Shallow trench isolation (STI) (or LOCOS in early processes, with feature size $> 0.25 \mu\text{m}$)
3. Well formation & Source and drain module formation.



1.4: BACK END OF IC FABRICATION:

The back end of line (BEOL) is the second portion of IC fabrication where the individual devices (transistors, capacitors, resistors, etc.) get interconnected with wiring on the wafer, the metalization layer. Common metals are Copper interconnect and Aluminum interconnect. BEOL generally begins when the first layer of metal is deposited on the wafer. BEOL includes contacts, insulating layers (dielectrics), metal levels, and bonding sites for chip-to-package connections.

After the last FEOL step, there is a wafer with isolated transistors (without any wires). In BEOL part of fabrication stage contacts (pads), interconnect wires, vias and dielectric structures are formed. For modern IC process, more than 10 metal layers can be added in the BEOL.

The process used to form DRAM capacitors creates a rough and hilly surface, which makes it difficult to add metal interconnect layers and still maintain good yield. In 1998, state-of-the-art DRAM processes had 4 metal layers, while state-of-the-art logic processes had 7 metal layers.

As of 2002, 5 or 6 layers of metal interconnect are common. As of 2009, typical DRAM devices (1 Gbit) use 3 layers of metal interconnect, tungsten on the first layer and aluminum on the higher layers. As of 2011, many gate arrays are available with a 3-layer interconnect. Many power ICs and analog ICs use a 3-layer interconnect. The top-most layers of a chip have the thickest and widest and most widely separated metal layers, which make the wires on those layers have the least resistance and smallest RC time delay, so they are used for power distribution and clock distribution. The bottom-most metal layers of the chip, closest to the transistors, have thin, narrow, tightly-packed wires, used only for local interconnect.

Adding layers can potentially improve performance, but adding layers also reduces yield and increases manufacturing cost. Chips with a single metal layer typically use the poly silicon layer to "jump across" when one signal needs to cross another signal such as the RCA CDP1802 and the 4004, giving effectively 2 layers of interconnections. Many microprocessors were designed with two metal interconnect layers, both of them aluminum, including the 1987CVAX and the 1989 rigel.

Many high-performance microprocessors were designed with 3 metal interconnect layers, all of them aluminum. Those included several processors using the process, including the 1992 Alpha 21064; and processors using theCMOS-6 process, including the 1996 StrongARM.

The AMD Athlon Thunderbird has 6 interconnect layers, the AMD Athlon Palomino has 7 interconnect layers, the AMD Athlon Thoroughbred A has 8 interconnect layers, and the AMD Athlon Thoroughbred B has 9 interconnect layers The Intel Xeon Dunnington has nine copper interconnect layers

1.5: Floor Planning:

Floor planning is nothing but taking layout information into account at early stages of the design process.

1.5.1: The floorplan-based design methodology

Planning layout at early stages may suggest valuable architectural modifications. Floorplanning fits very well in a top-down design strategy, this stepwise refinement strategy is also propagated in software design. Floorplanning gives flexibility in layout design, the existence of cells that can adapt their shapes and terminal locations to the environment.

This is the first major step in getting the layout done, and is the most important one. Floorplan determines the chip quality. At this step, we decide the size of our chip/block, allocates power routing resources, place the hard macros, and reserve space for standard cells. Every subsequent stage like placement, routing and timing closure is dependent on how good the floorplan is. In a real time design, many iterations are performed before arriving at an optimum floorplan.

Core Boundary

Floorplan defines the size and shape of the chip/block. A top level digital design will have a rectangular/square shape, whereas a sub block may have rectangular or rectilinear shapes. Core boundary refers to the area where one will be placing standard cells and other IP blocks. One may have power routing spaces allocated outside the core boundary. For a full chip, we also have IO buffers and IO pads placed outside the core boundary.

In PnR tool, floorplanning can be controlled by various parameters:

Aspect ratio: This is the ratio of height divided by width and determines whether we get a square or rectangular floorplan. An aspect ratio of 1 gives a square floorplan.

Core utilization: $\text{Core utilization} = (\text{standard cell area} + \text{macro cells area}) / \text{total core area}$
A core utilization of 0.8 means that 80% of the area is available for placement of cells, whereas 20% is left free for routing.

Boundary: We need to specify a boundary and the tool can honour it. This comes in handy when we have an existing boundary from a previous version. When we specify Boundary as the control parameter, both aspect ratio and core utilization are irrelevant. The tool gives a report of the utilization for the current boundary specified.

IO Placement/Pin placement

In a digital-top design, we need to place IO pads and IO buffers of the chip. Take a rectangular or square chip that has pads in four sides. To start with, one may get the sides and relative positions of the PADs from the designers. We also get a maximum and minimum die size according to the package we have selected. Perl script is used to place IO once the chip size is decided.

Macro placement

Once the size & shape of the floorplan is ready and initialized, thereby creating standard cell rows, then place the macros. Do not use any auto placement. Flylines in the tool will show the connection between the macros and standard cells or IOs.

- i. Use flylines and make sure to place blocks that connects each other closer
- ii. For a full-chip, if hard macros connect to IOs, place them near the respective IOs.
- iii. Consider the power straps while placing macros.

4. Creating Power Rings and Straps

Generating the power rings using IC Compiler can be done as follows:

First decide the trunks that supply power to the core then make sure that all the hard macros have sufficient rings/straps around it to hook into the PG trunks. As usual, a robust power structure will take iterations and IR drop analysis at a later stage, but a close approximation can be arrived at the initial stages.

1.5.2 Floorplanning concepts

1.5.2.1 Abutment: **Establishing connections between cells by putting them directly next to each other, without the necessity of routing as shown in figure 3.12.2.1**

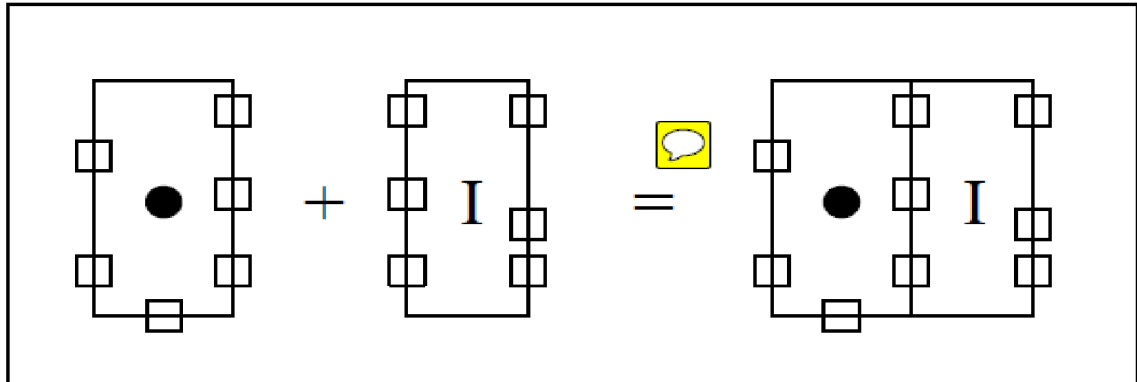


Figure 1.5.2.1: Abutment

1.5.2.2 Leaf cell:

A cell at the lowest level of the hierarchy, it does not contain any other cell.

1.5.2.3 Composite cell:

A cell that is composed of either leaf cells or composite cells. The whole chip is the highest-level composite cell.

1.5.2.4 Slicing floorplans:

A floorplan with the property that a composite cell's subcells are obtained by a horizontal or vertical bisection of the composite cell. Slicing floorplans can be represented by a slicing tree.

In a slicing tree, all cells (except for the toplevel cell) have a parent, and all composite cells have children. Not all floorplans are slicing. Limiting floorplans to those that have the slicing property is reasonable. It certainly facilitates floorplanning algorithms. Sliced floorplan is as shown in figure 3.12.2.2.

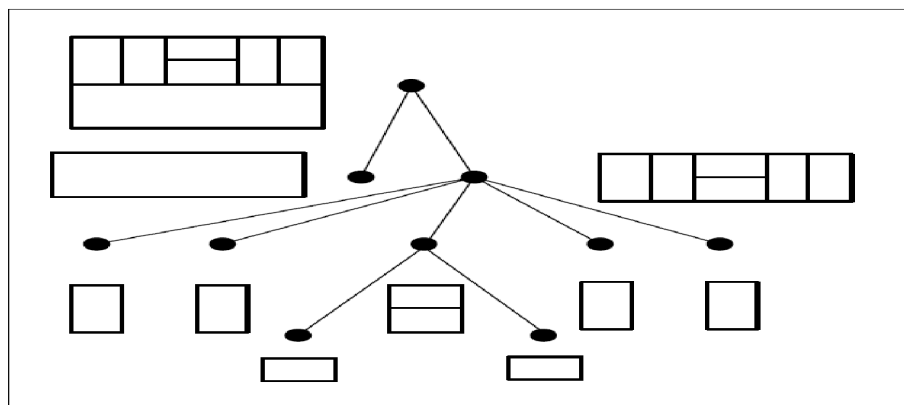


Figure 1.5.2.2: Siliced floorplan

1.6: PLACEMENT:

Placement does not just place the standard cells available in the synthesized netlist. It also optimizes the design, thereby removing any timing violations created due to the relative placement on die.

1.6.2: Important things in placement:

1. High fanout net synthesis

High fanout nets other than clocks are synthesized at the placement stage. In logic synthesis, high fanout nets like reset, scan enable etc are not synthesized. The SDC used for PnR should not have any `set_ideal_network` or `set_dont_touch` commands on these signals. Also, make sure to set an appropriate fanout limit for your library using the command `set_max_fanout`.

If a driver has too many loads, it will negatively affect the delay numbers and transitions values. After placement, check for any fanout violations in the timing report.

2. Use Ideal clock

Synthesize the clock later in the design. Define the clocks as ideal. If not, HFN synthesis will be done on the clock. Clock constraints like skew or clock buffers are not used, and effectively your clock tree is messed up.

Control Congestion

Congestion needs to be analyzed after placement and the routing results depend on how congested the design is. Routing congestion may be localized. Some of the things that can be done to make sure routing is hassle free are:

Macro-padding:

Macro padding or placement halos around the macros are placement blockages around the edge of the macros. This makes sure that no standard cells are placed near the pin outs of the macros, thereby giving extra breathing space for the macro pin connections to standard cells.

5. Maximum Utilization constraint:

Some tools specifies maximum core utilization numbers for specific regions. If any region has routing congestion, utilization there can be reduced, thus freeing up more area for routing.

Placement blockages:

The utilization constraint is not a hard rule, and if you want to specifically avoid placement in certain areas, use placement blockages.

Scan chain reordering:

In a less complex design, usually scan reordering is carried out. However, sometimes it may become difficult to pass scan timing constraints once the placement is done. The scan flip flop placements may create lengthier routes if the consecutive flops in scan chain are placed far apart due to a functional requirement. In this case, the PnR tool can reconnect the scan chains, to make routing easier. A prerequisite for this option is a scan DEF for the tool to recognize the chains.

TIE cells

In your netlist, some unused inputs are tied to either VDD/VSS (or logic1/logic0). It is not recommended to connect a gate directly to the power network,

so one can use TIEHI or TIELO cells if available in the library for the same. These are single pin cells which effectively ties the pin it connects high or low. After placement, dump out a netlist and search for direct pin connections to the PG rails (other than power pins).

1.7: Clock Tree Synthesis

Clock Tree Synthesis is a process which makes sure that the clock gets distributed evenly to all sequential elements in a design. The goal of CTS is to minimize the skew and latency. The placement data will be given as input for CTS, along with the clock tree constraints. The clock tree constraints will be Latency, Skew, Maximum transition, Maximum capacitance, Maximum fan-out, list of buffers and inverters etc.

The clock tree synthesis contains clock tree building and clock tree balancing. Clock tree can be build by clock tree inverters so as to maintain the exact transition (duty cycle) and clock tree balancing is done by clock tree buffers (CTB) to meet the skew and latency requirements. Less clock tree inverters and buffers should be used to meet the area and power constraints.

Standard Clock Tree Synthesis engines are driven by timing closure and, hence, are not PVT (process/voltage/temperature) variation aware. They are used to fix setup/hold violations by adjusting the clock skew, adding, removing, and swapping buffers, or exploiting different clock wire lengths and levels and so on. As a result, the skew sensitivity with respect to PVT variations cannot be kept low, since there are several contributors originating from different physical phenomena.

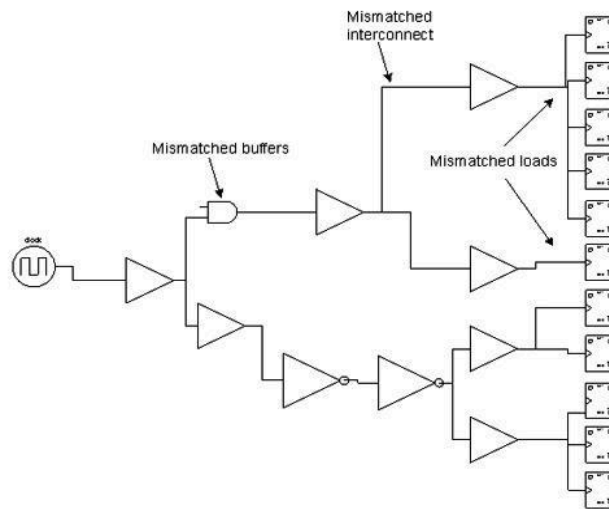


Figure 1.7.3: Randomly connected Buffer

One approach to overcoming the impairment of clock skew in chip design, mostly in high speed design, is the clock MESH, as schematized. The major difference between this method and the standard approach to clock tree synthesis (CTS) is that at a certain level of the tree, the drivers' outputs are connected to the same metal net, called a mesh net. Such a shorting of several clock drivers enables an averaging and spatial smoothing effect, which reduces the clock skew of the different clock drivers.

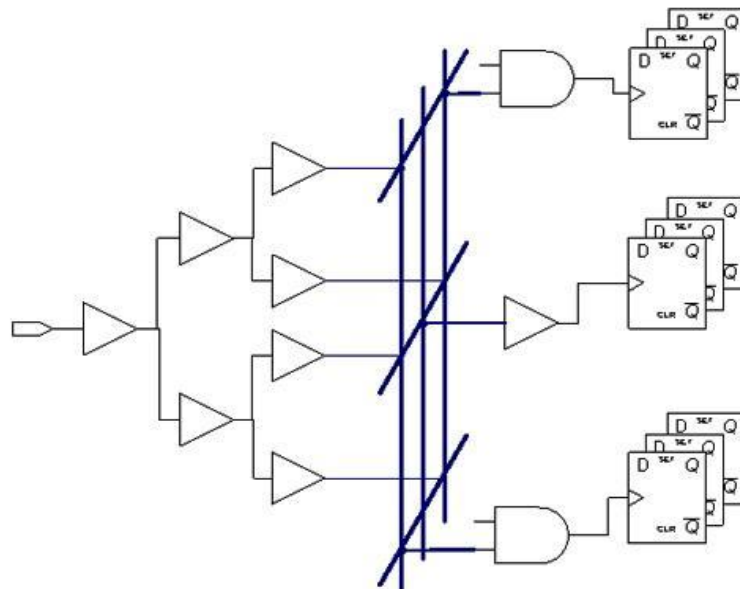


Figure 1.7.3.2: Orderly Connected Buffers.

Clock mesh technology produces a much lower clock skew compared to a conventional clock tree. Unfortunately, one issue is how to take full advantage of this technique in the standard design flow. Typically a full set of analog simulations are needed to evaluate the residual clock uncertainty on the mesh net before continuing with the timing analysis. A correct skew evaluation needs the layout to be frozen and every alteration of the pre-mesh structure obliges this out-of-the-flow characterization to be run again, making the overall design flow very long.

Another problem with a clock mesh, especially when conceived at top level, is the large amount of power consumed, which means that a dynamic power drop is likely. Other techniques found in the literature, such as PLL/DLL de-skewing, are not suited for high precision, low uncertainty clock distribution, mainly due to added jitter of PLL/DLL circuitry.

1.7.3: Setup and Hold Fixing

Set Up Fixing:

- I. Upsizing the cells (increase the drive strength) in data path.
- II. Pull the launch clock
- III. Push the capture clock
- IV. We can reduce the buffers from datapath .
- V. We can replace buffers with two inverters placing farther apart so that delay can adjust.
- VI. We can also reduce some larger than normal capacitance on a cell output pin.
- VII. We can upsize the cells to decrease the delay through the cell.
- VIII. LVT cells

Hold Fixing:

It is well understood hold time will be large if data path has more delay. So we have to add more delays in data path.

- I. Downsizing the cells (decrease the drive strength) in data path.
- II. Pulling the capture clock.
- III. Pushed the launch clock.
- IV. By adding buffers/Inverter pairs/delay cells to the data path.
- V. Decreasing the size of certain cells in the data path, It is better to reduce the cells in capture path closer to the capture flip flop because there is less chance of affecting other paths and causing new errors.
- VI. By increasing the wire load model, we can also fix the hold violation.

1.8: ROUTING

The routing process determines the precise paths for interconnections. This includes the standard cell and macro pins, the pins on the block boundary or pads at the chip boundary. After placement and CTS, the tool has information about the exact locations of blocks, pins of blocks, and I/O pads at chip boundaries. The logical connectivity as defined by the netlist is also available to the tool. In routing stage, metal and vias are used to create the electrical connection in layout so as to complete all connections defined by the netlist. Now, to do the actual interconnections, the tool relies on some “Design Rules”.

Most of the routers available are grid based routers. There are routing grids defined for the entire layout. Consider it like a graph as below. For grid based routers, there are also preferred routing directions defined for each metal layer. e.g. Metal1 has a preferred direction of “horizontal”, metal2 has preferred routing direction of “vertical”

and so on. So, in the whole layout, metal1 routing grids will be drawn (superimposed) horizontally with metal1 wire pitch and metal2 grids will be drawn vertically with metal2 wire pitch between each.

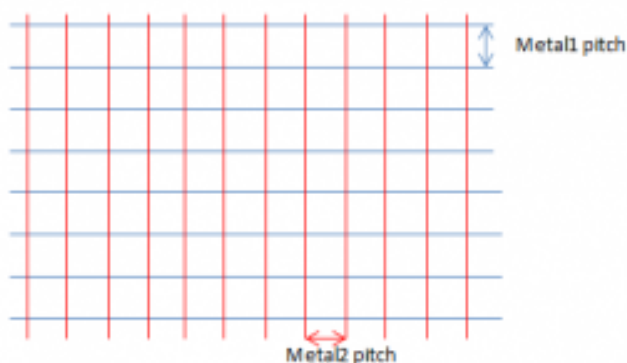


Figure 1.8.1 Routing Grids

The above figure shows how routing grids are drawn. Here only two metals are considered for now, but in a process with more metals, similar grids will be superimposed on the layout for all available metals. Pitch is calculated by determining the minimum spacing required between grid lines of same metal.

This can be the minimum spacing of the metal itself, but is usually a value greater than the minimum spacing. This is calculated by taking into account the via dimension as well, so that no two adjacent wires on the grid create any DRC violation even when there are vias present.

1.4.1: Routing Congestion

It is difficult to route a highly congested design. Some not-so congested designs may have pockets of high congestion which will again create routing issues. It is important that the congestion is analyzed and fixed before detailed routing. After CTS, the tool can give you a congestion map by a trial route/ global route values.

1.4.2: Routing Order

It is recommended that you route sensitive nets like clock before the rest of the signal route. My assumption is that you have completed power routing after the floorplan stage (because that is what I do.). For this discussion I am going with a traditional routing approach and not considering signal integrity issues. The order of routing is:

1.4.2.1: Power Routing:

Connect the macro and standard cell power pins to the power rings and staps you have created for the design. IR drop

1.4.2.2: Clock Routing:

We do not want to upset the skew and delay values for the clock net as much as possible. So the clocks are given higher priority in using routing resources and routed prior to any other net routing. Clock routing can be limited to higher metal layers for reduced RC numbers.

1.4.2.3: Signal Routing:

The rest of the nets are routed. We can also route groups of nets, and non-default routing rules can also be applied to select nets.

1.9: Signal Integrity

By definition, “integrity” means “complete and unimpaired.” Likewise, a digital signal with good integrity has clean, fast transitions; stable and valid logic levels; accurate placement in time and it would be free of any transients.

Evolving technology makes it increasingly difficult for system developers to produce and maintain complete, unimpaired signals in digital systems.

The term Signal Integrity (SI) addresses two concerns in the electrical design aspects – the timing and the quality of the signal. The goal of signal integrity analysis is to ensure reliable high-speed data transmission. In a digital system, a signal is transmitted from one component to another in the form of logic 1 or 0, which is actually at certain reference voltage levels. At the input gate of a receiver, voltage above the reference value V_{IH} (high input voltage) is considered as logic high, while voltage below the reference value V_{IL} (low input voltage) is considered as logic low.

Figure 1.9.1 shows the ideal voltage waveform in the perfect logic world, whereas Figure 1.9.2 shows how signal will look like in a real system. More complex data, composed of a string of bits 1s and 0s, are actually continuous voltage waveforms.

The receiving component needs to sample the waveform in order to obtain the binary encoded information. The data sampling process is usually triggered by the rising edge or the falling edge of a clock signal as shown in the Figure 3.16.3. It is clear from the diagram that the data must arrive at the receiving gate on time and settle down to a non-ambiguous logic state when the receiving component starts to latch in. Any delay of the data or distortion of the data waveform will result in a failure of the data transmission. If the signal waveform in Figure 3.13.2.1 exhibits excessive ringing into the logic gray zone while the sampling occurs, then the logic level cannot be reliably detected.

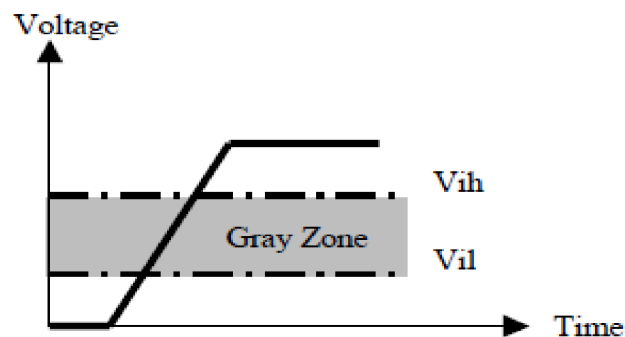


Figure 1.9.1: Ideal Waveform at the Receiving Gate.

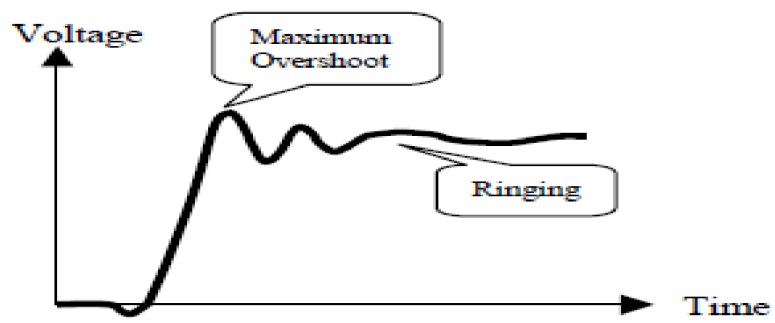


Figure 1.9.2: Real Waveform at the Receiving Gate.

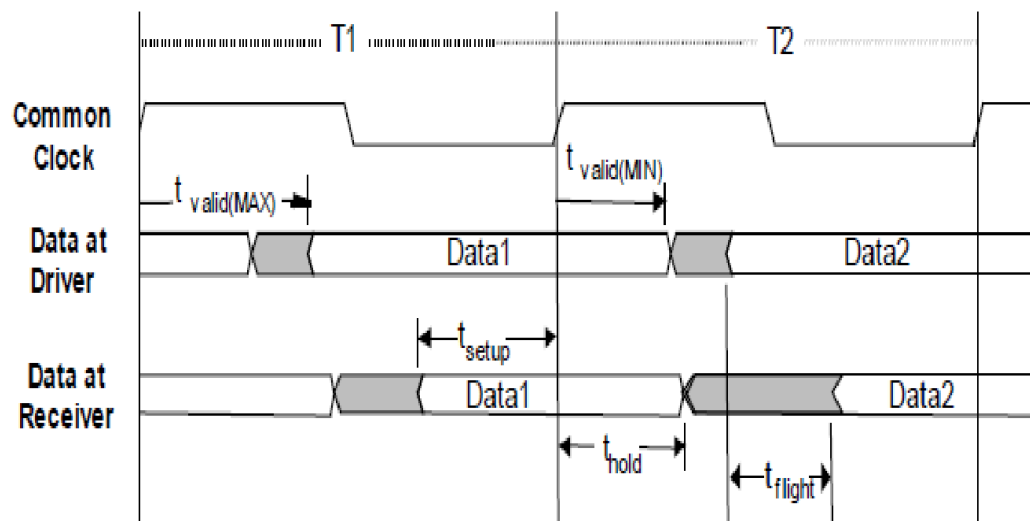


Figure 1.9.3: Data Sampling Process and Timing Conventions.

2.1: The MOSFET as A Switch:

MOSFET is a voltage controlled field effect transistor, it has a “Metal Oxide” Gate electrode which is electrically insulated from the main semiconductor n-channel or p-channel by a very thin layer of insulating material usually silicon dioxide, commonly known as glass.

This ultra thin insulated metal gate electrode can be thought of as one plate of a capacitor. The isolation of the controlling Gate makes the input resistance of the **MOSFET** extremely high way up in the Mega-ohms ($M\Omega$) region thereby making it almost infinite.

As the Gate terminal is isolated from the main current carrying channel “NO current flows into the gate, the MOSFET also acts like a voltage controlled resistor were the current flowing through the main channel between the Drain and Source is proportional to the input voltage. MOSFETs are three terminal devices with a Gate, Drain and Source and both P-channel (PMOS) and N-channel (NMOS) MOSFETs are available.

2.2: MOSFET Characteristics Curves

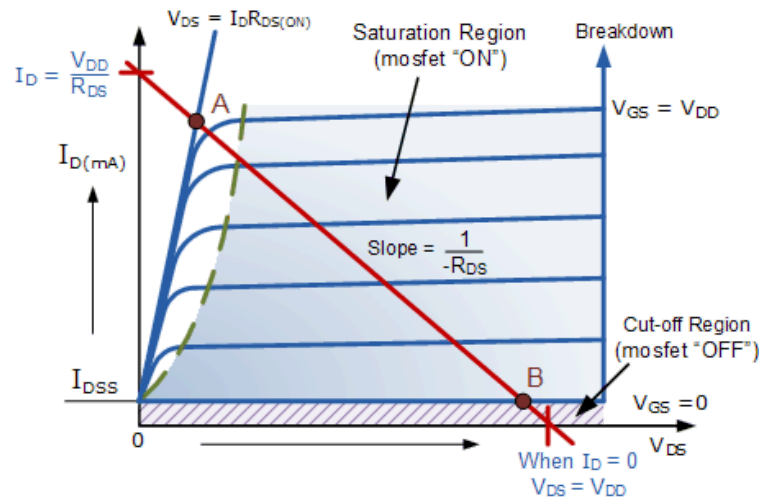


Figure 2.2.1: MOSFET characteristic curves.

The minimum ON-state gate voltage required to ensure that the MOSFET remains “ON” when carrying the selected drain current can be determined from the V-I transfer curves above in figure 2.2.1.1. When V_{IN} is HIGH or equal to V_{DD} , the MOSFET Q-point moves to point A along the load line. The drain current I_D increases to its maximum value due to a reduction in the channel resistance. I_D becomes a constant value independent of V_{DD} , and is dependent only on V_{GS} . Therefore, the transistor behaves like a closed switch but the channel ON-resistance does not reduce fully to zero due to its $R_{DS(on)}$ value, but gets very small.

Likewise, when V_{IN} is LOW or reduced to zero, the MOSFET Q-point moves from point A to point B along the load line. The channel resistance is very high so the transistor acts like an open circuit and no current flows through the channel. So if the gate voltage of the MOSFET toggles between two values, HIGH and LOW the MOSFET will behave as a “single-pole single-throw” (SPST) solid state switch. The MOSFET basically has two regions, cut-off region and saturation region.

2.3: DIGITAL-CIRCUIT SPEED AND POWER CONSUMPTION

The essential figures of merit of a digital circuit or system are speed and power consumption. The usual measure for speed is a (reciprocal) delay time t_d or a maximum clock frequency $f_{c,max}$. Power efficiency can be determined as the total power P_{tot} or in terms of a switching energy E_s , i.e., the average energy consumed for one switching transition of a device.

Consider a digital circuit as a network of switches with parasitic capacitances, which can be described statistically in terms of average values. Thus, one representative pair switches that charge and discharge a representative load capacitance C_L to the supply voltage V_{DD} with a current I_{on} . Each switch in the off-state draws a leakage current I_{off} . For simplicity, any short-circuit current flowing across the switches is assumed to be negligible (which is usually acceptable), and a pair of switches is regarded as one device.

The total power consumption per device is the sum of a dynamic component from charging and discharging the capacitance and a static component from the leakage current:

$$P_{tot} = P_{dyn} + P_{stat} = \alpha f_c C_L (V_{DD})^2 + I_{off} V_{DD}$$

In this expression f_c is the clock frequency and α is the switching probability, the so-called activity ratio. A more universal measure is the switching energy which is related to the power consumption as seen in equation

$$E_s = C_L (V_{DD})^2 + (1/\alpha f_c) I_{off} V_{DD}$$

Therefore

$$P_{tot} = \alpha f_c E_s$$

The advantage of using the switching energy rather than the total power consumption is that E_s is independent of the system throughput. For example, when pipelined architectures are considered the power consumption varies with the architectural changes, but the switching energy does not and is therefore the better candidate to optimize.

The speed of a digital circuit can be characterized in two ways: the delay time t_d which is assumed as and the maximum clock frequency $f_{c,max}$ which is given by

$$T_d = C_L V_{DD} / I_{on}$$

$$f_{c,max} = 1 / t_d l_d$$

In the equation 3.4.5 the logic depth l_d is the number of stages through which a switching event must propagate during one clock cycle.

Clearly, a system is most efficient when operated at the maximum clock frequency. assuming that $f_c = f_{c,max}$ yields the following fundamental equation:

$$E_s = C_L (V_{DD})^2 [1 + (l_d / \alpha) (I_{off} / I_{on})]$$

The following two conclusions can be drawn from :

1. The switching energy increases quadratically with the supply voltage.
2. The leakage current I_{off} does not increase the switching energy as long as $I_{off} / I_{on} \ll \alpha / l_d$. Thus, the key to reducing the power consumption is to reduce the supply voltage. Typical values of α / l_d for a microprocessor are 0.1/7, i.e., the leakage current can be almost as large as $I_{on} / 70$ which is several orders of magnitude above the conventional leakage constraints.

2.4: MOS LOGIC

MOS: Metal Oxide Semiconductor

Transistors are built on a **Silicon** (semiconductor) substrate. Pure silicon has no free carriers and conducts poorly. Dopants are added to increase conductivity: extra electrons (**n-type**) or extra holes (**p-type**).

MOS structure is created by superimposing several layers of conducting, insulating and transistor forming materials. Metal gate has been replaced by polysilicon or poly in modern technologies.

There are two types of MOS transistors:

nMOS : Negatively doped silicon, rich in electrons.

pMOS : Positively doped silicon, rich in holes.

CMOS: Both type of transistors are used to construct any gate.

2.4.1: Signal Strengths

Signals such as **1** and **0** have strengths, measures ability to sink or source current. VDD and GND rails are the strongest 1 and 0. Under the switch abstraction, G has complete control and S and D have no effect.

In reality, the gate can turn the switch ON only if a potential difference of at least V_t , that exists between the G and S. Thus signal strengths are related to V_t and therefore p and n transistors produce signals with different strengths such as:

Strong 1: VDD,

Strong 0: GND,

Weak 1: $(\sim VDD - V_t)$ and

Weak 0: $(\sim GND + V_t)$.

The different signal strengths of nMOS and pMOS are shown in figure 3.5.1

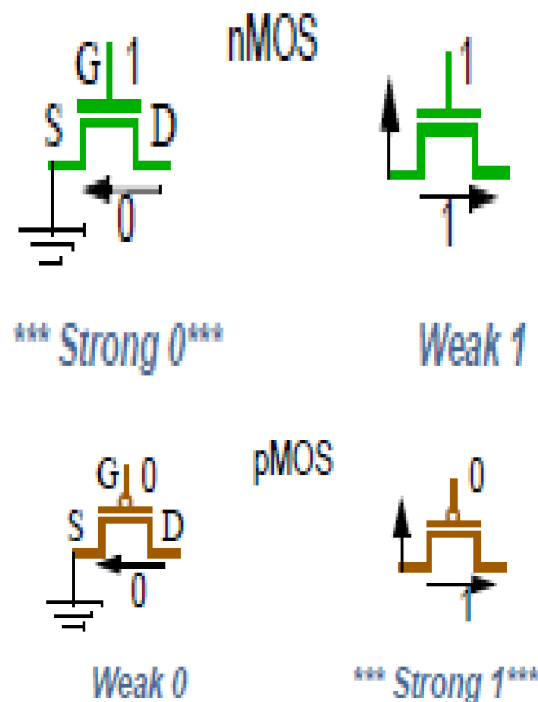


Figure 2.4.1: Different signal strengths seen in nmos and pmos

2.5: COMBINATIONAL & SEQUENTIAL LOGIC

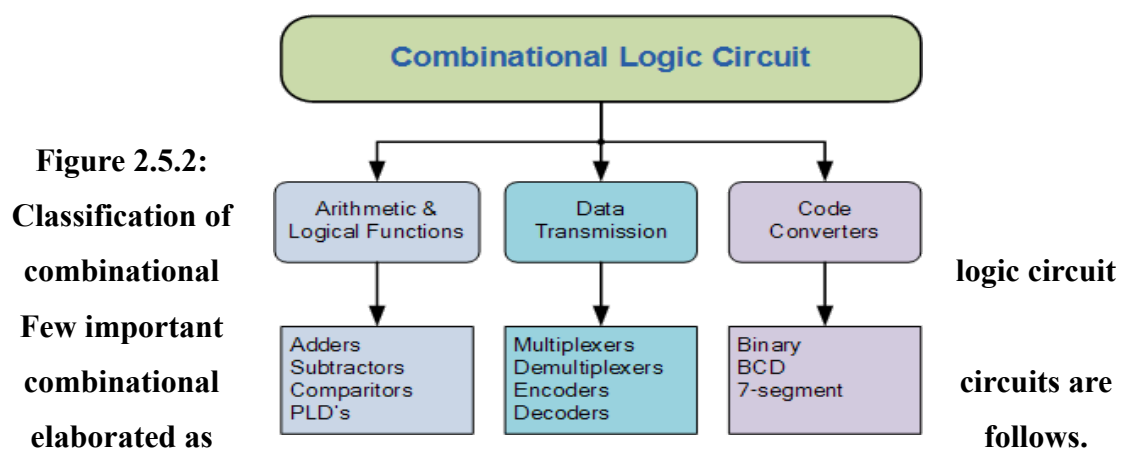
Combinational circuit is a circuit in which we combine the different gates in the circuit, for example encoder, decoder, multiplexer and demultiplexer. Some of the characteristics of combinational circuits are following:-

- The output of combinational circuit at any instant of time depends only on the levels present at input terminals.
- The combinational circuits do not use any memory. The previous state of input does not have any effect on the present state of the circuit.
- A combinational circuit can have an n number of inputs and m number of outputs as shown in figure 3.3.2 and its classification can be seen in figure 3.3.2.1.

2.5.1 combinational logic representation:



Figure 2.5.1: Block diagram of combinational circuit



Examples of combinational circuits are Half Adder, Full Adder, Multiplexers, Demultiplexer, Decoder, encoder etc.

The output state of a “sequential logic circuit” is a function of the following three states, the “present input”, the “past input” and/or the “past output”. Sequential Logic circuits remember these conditions and stay fixed in their current state until the next clock signal changes one of the states, giving sequential logic circuits “Memory”.

Sequential logic circuits are generally termed as two state or **Bistable** devices which can have their output or outputs set in one of two basic states, a logic level “1” or a logic level “0” and will remain “latched” (hence the name latch) indefinitely in this current state or condition until some other input trigger pulse or signal is applied which will cause the bistable to change its state once again.

2.5.2: Sequential Logic Representation

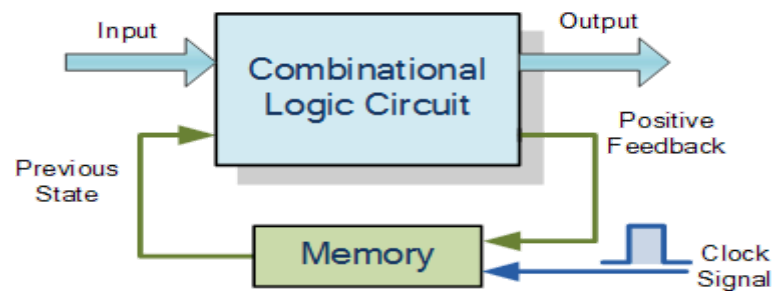


Figure 2.5.2.1: Representation of sequential logic circuit

The word “Sequential” means that things happen in a “sequence”, one after another and in **Sequential Logic** circuits, the actual clock signal determines when things will happen next. Simple sequential logic circuits can be constructed from standard **Bistable** circuits such as: Flip-flops, Latches and Counters and which themselves can be made by simply connecting together universal **NAND Gates** and/or **NOR Gates** in a particular combinational way to produce the required sequential circuit. Figure 4.5.2.1 shows the representation of sequential logic circuit.

2.5.3: Classification of Sequential Logic

As standard logic gates are the building blocks of combinational circuits, bistable latches and flip-flops are the basic building blocks of Sequential Logic Circuits. Sequential logic circuits can be constructed to produce either simple edge-triggered flip-flops or more complex sequential circuits such as storage registers, shift registers, memory devices or counters. Either way sequential logic circuits can be divided into the following three main categories:

1. Event Driven – asynchronous circuits that change state immediately when enabled.
2. Clock Driven – synchronous circuits that are synchronized to a specific clock signal.
3. Pulse Driven – This is a combination of the two that responds to triggering pulses.

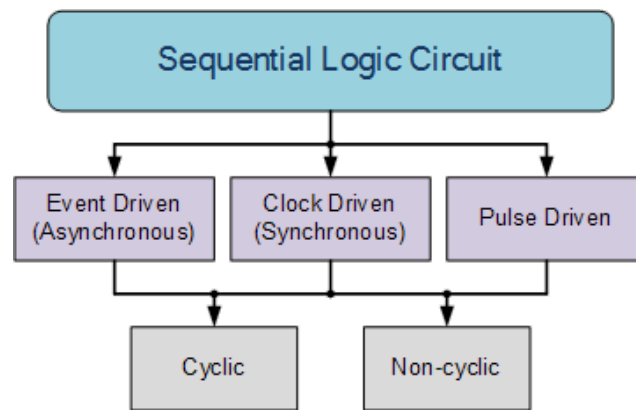


Figure 2.5.3.1: Classification of sequential logic circuits

Sequential logic circuits return back to their original steady state once reset and sequential circuits with loops or feedback paths are said to be “cyclic” in nature. Sequential circuits changes occur only on the application of a clock signal making it synchronous, otherwise the circuit is asynchronous and depends upon an external input. To retain their current state, sequential circuits rely on feedback and this occurs when a fraction of the output is fed back to the input. The classification of sequential circuits is seen in figure 2.3.3.2.

2.3.4: Synchronous sequential circuit

A master-clock generator is used to generate a periodic train of clock pulses. These clock pulses are distributed throughout the system. Clocked sequential circuits are most commonly used. The memory elements are flip-flops. These flip-flops are the binary cells capable of storing one bit of information. It generates two outputs: one for the normal value and one for the complement value. It also maintains a binary state indefinitely until directed by an input signal to switch states.

4.6: SYNCHRONIZATION

Data Synchronization is a process of establishing consistency among systems and subsequent continuous updates to maintain consistency. It should not be considered as a

one-time task. It is really a process which needs to be planned, owned, managed, scheduled and controlled.

2.3.2: Process of synchronization

1) Planning

Requirements on the data synchronization should be gathered in the planning phase. It covers the data content, data formats, initial load and frequency of the updates. Non-functional requirements like performance, timing and security are covered as well.

2) Ownership

Although the data synchronization idea may come from the IT organization of the company, an owner or champion from the company business is necessary to provide a continuity of the initiative. It is business who will benefit from the data synchronization initiative in the end.

3) Scheduling

The scheduling and frequency of updates is one of the items which needs to be investigated during initial planning phase. Often the requirements change during this time and the schedule updates needs to be revised. Obviously, the granularity of a schedule on which the updates/synchronization is performed cannot be finer than the source system is able to provide. However, the scheduling also needs to take into account performance aspects.

4) Monitoring

The synchronization process should be monitored to evaluate whether the update schedule and frequency meets the company's needs.

From the technical point of view, the synchronization may be implemented on any level:

- ☐ System/Application level
- ☐ File level
- ☐ Record level synchronization

2.3.2.1: Challenges of synchronization

1) Data Formats Complexity

As the enterprise grows and evolves, new systems from different vendors are implemented. The data formats for employees, products, suppliers and customers vary among different industries which results not only in building a simple interface between the two applications (source and target), but also in a need to transform the data while passing them to the target application. The data formats, of course, vary from proprietary formats through plain text to xml. Some of the applications provide API to push the data directly. ETL tools can be helpful here.

2) Real-timeliness

The requirement today is that the systems are real time. Customers want to see what the status of their order in e-shop is; the status of a parcel delivery - a real time parcel tracking; what the current balance on their account is; etc. Enterprises need to have their system real-time updated as well to enable smooth manufacturing process, e.g. ordering material when enterprise is running out stock; synchronizing customer orders with manufacturing process, etc. There are thousands of examples from real life when the real time is becoming either advantage or a must to be successful and competitive.

3) Security

Different systems may have different policies to enforce data security and access levels. Even though the security is maintained correctly in the source system which captures the data, the security and information access privileges must be enforced on the target systems as well to prevent any potential misuse of the information. This is particularly an issue when handling personal information or any piece of confidential information under Non Disclosure Agreement (NDA). Any intermediate results of the data transfer as well as the data transfer itself must be encrypted.

4) Data Quality

Maintaining data in one place and sharing with other applications is best practice in managing and improving data quality. This prevents inconsistencies in the data caused by updating the same data in one system.

5) Performance

The data synchronization process consists basically of five phases:

1. Data extraction from the source/master system
2. Data transfer
3. Data transformation
4. Data transfer
5. Data load to the target system

In case of large data, each of these steps may impact performance. Therefore, the synchronization needs to be carefully planned to avoid any negative impact e.g. during peak processing hours.

6) Maintenance

As any other process, the synchronization process needs to be monitored to ensure that it is running as scheduled and properly handling any errors during the process of synchronization such as rejected records or malformed data.

2.6.3: Setup & Hold Time

Every flip-flop has restrictive time regions around the active clock edge in which input should not change. Any change in the input in this region, the output may be derived from either the old input, the new input, or even in between the two. Here we define, two very important terms in the digital clocking, Setup and Hold time.

- The **setup time** is the interval before the clock where the data must be held stable.
- The **hold time** is the interval after the clock where the data must be held stable. Hold time can be negative, which means the data can change slightly before the clock edge and still be properly captured. Most of the current day flip-flops have zero or negative hold time.

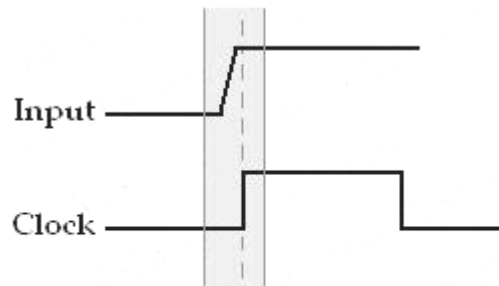


Figure 2.6.3.1: Waveform Demonstrating Set-Up and Hold Time

In the above figure, the shaded region is the restricted region. The shaded region is divided into two parts by the dashed line. The left hand side part of shaded region is the setup time period and the right hand side part is the hold time period. If the data changes in this region, as shown in the figure 2.6.3.1, the output may, follow the input, or may not follow the input, or may go to metastable state (where output cannot be recognized as either logic low or logic high, the entire process is known as metastability).

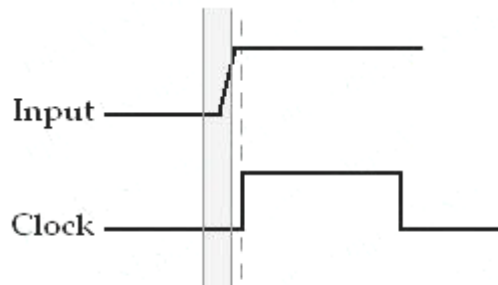


Figure 2.6.3.2: Waveform Demonstrating Set-Up time

Figure 2.6.3.2 shows the restricted region (shaded region) for a flip-flop whose hold time is negative. The diagram in figure 2.6.3.3 illustrates the restricted region of a D flip-flop. D is the input, Q is the output, and clock is the clock signal. If D changes in the restricted region, the flip-flop may not behave as expected, means Q is unpredictable.

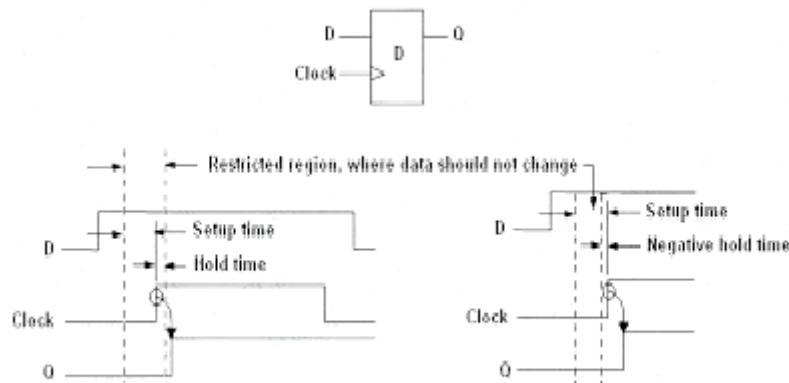


Figure 2.6.3.3: Timing Diagram Of D-FlipFlop

To avoid setup time violations:

- The combinational logic between the flip-flops should be optimized to get minimum delay.
- Redesign the flip-flops to get lesser setup time.
- Tweak launch flip-flop to have better slew at the clock pin, this will make launch flip-flop to be fast there by helping fixing setup violations.
- Play with clock skew (useful skews).

To avoid hold time violations:

- By adding delays (using buffers).
- One can add lockup-latches (in cases where the hold time requirement is very huge, basically to avoid data slip).

2.6.4: Metastability

Whenever there are setup and hold time violations in any flip-flop, it enters a state where its output is unpredictable: this state is known as metastable state (quasi stable state); at the end of metastable state, the flip-flop settles down to either '1' or '0'. This whole process is known as metastability. In the figure 2.3.2 below T_{su} is the setup

time and T_h is the hold time. Whenever the input signal D does not meet the T_{su} and T_h of the given D flip-flop, metastability occurs.

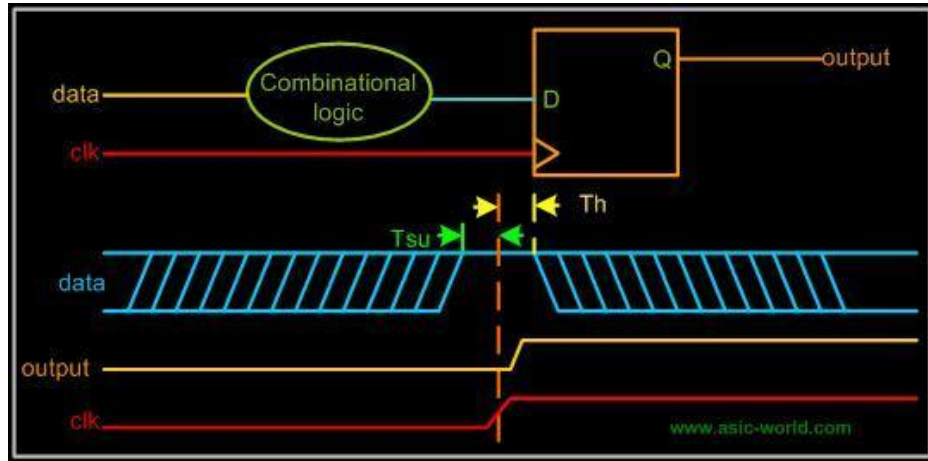


Figure 2.6.4.1: Example Demonstrating Timing Constraints.

When a flip-flop is in metastable state, its output oscillates between '0' and '1' as shown in the figure 2.6.4.1 (here the flip-flop output settles down to '0'). The time taken to settle down, depends on the technology of the flip-flop.

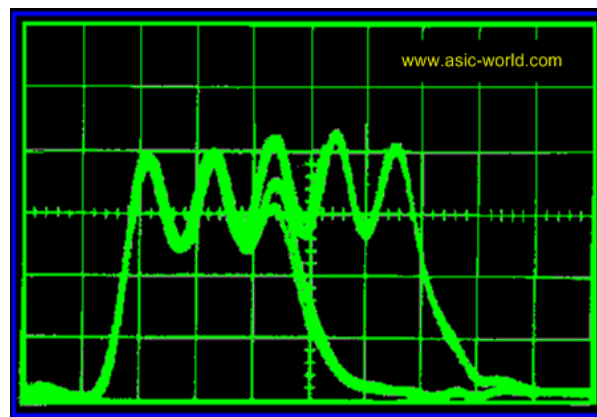


Figure 2.6.4.2: Metastable State.

If we look deep inside of the flip-flop we see that the quasi-stable state is reached when the flip-flop setup and hold times are violated. Assuming the use of a positive edge triggered "D" type flip-flop, when the rising edge of the flip-flop clock occurs at a point in time when the D input to the flip-flop is causing its master latch to transition, the flip-flop is highly likely to end up in a quasi-stable state. This rising clock causes

the master latch to try to capture its current value while the slave latch is opened allowing the Q output to follow the "latched" value of the master. The most perfectly "caught" quasi-stable state (on the very top of the hill) results in the longest time required for the flip-flop to resolve itself to one of the stable states as shown in figure 2.6.4.3.

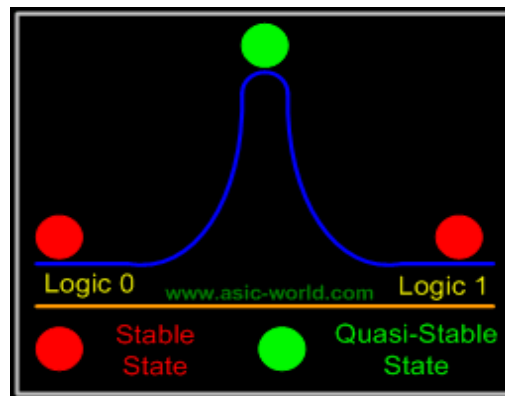


Figure 2.6.4.3: Quasi Stable State.

The relative stability of states in the figure 2.6.4.3 above shows that the logic 0 and logic 1 states (being at the base of the hill) are much more stable than the somewhat stable state at the top of the hill. In theory, a flip-flop in this quasi-stable hilltop state could remain there indefinitely but in reality it won't. Just as the slightest air current would eventually cause a ball on the illustrated hill to roll down one side or the other, thermal and induced noise will jostle the state of the flip-flop causing it to move from the quasi-stable state into either the logic 0 or logic 1 state.

Causes of metastability:

When the setup time and hold time violation occurs, then metastability occurs, so we have to see when signals violate this timing requirement:

- When the input signal is an asynchronous signal.
- When the clock skew/slew is too much (rise and fall time are more than the tolerable values).
- When interfacing two domains operating at two different frequencies or at the same frequency but with different phase.

- When the combinational delay is such that flip-flop data input changes in the critical window (setup + hold window).

4.7: ADVANCED LOW POWER TECHNIQUES

Power consumption can be divided into two aspects:

1) Dynamic power – the power that is consumed by a device when it is actively switching from one state to another. Dynamic power consists of switching power, consumed while charging and discharging the loads on a device, and internal power (also referred to as short circuit power), consumed internal to the device while it is changing state.

2) Leakage power – the power consumed by a device not related to state changes (also referred to as static power). Leakage power is actually consumed when a device is both static and switching, but generally the main concern with leakage power is when the device is in its inactive state, as all the power consumed in this state is considered “wasted” power.

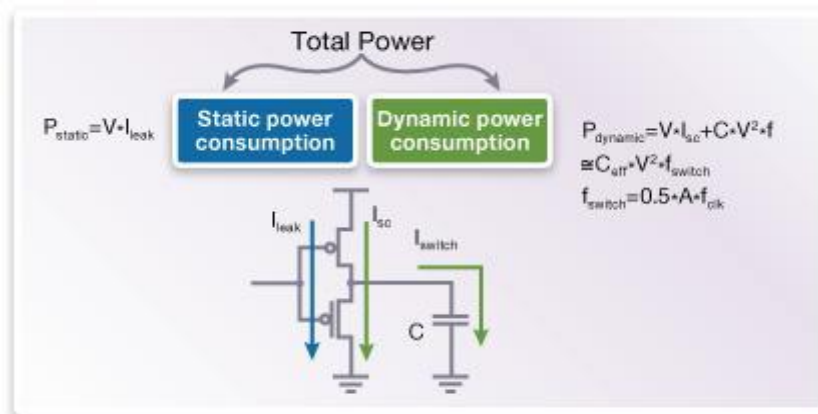


Figure 2.3.3: Total Power Consumption.

2.3.3: Techniques to reduce dynamic and leakage power

Various techniques have been developed to reduce both dynamic and leakage power. The two most common traditional techniques are:

1) Clock gating – involves disconnecting of the clock from a device it drives when the data going into the device is not changing. This technique is used to minimize dynamic power.

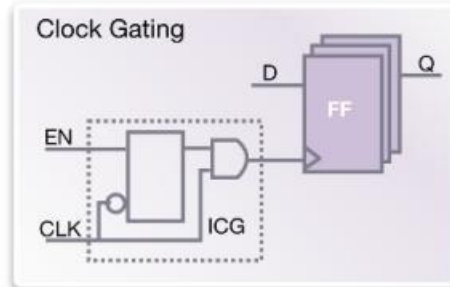


Figure 2.3.3.1: Clock Gating.

2) Multi-V_{th} optimization: involves replacement of faster Low-V_{th} cells, which consume more leakage power, with slower High-V_{th} cells, which consume less leakage power. Since the High-V_{th} cells are slower, this swapping only occurs on timing paths that have positive slack and thus can be allowed to slow down.

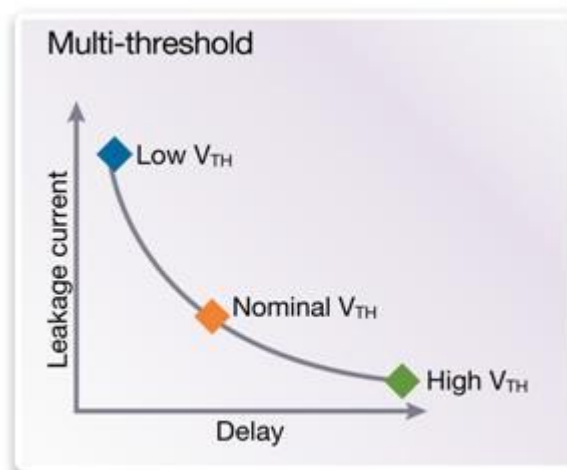


Figure 2.3.3.2: Multi V_{th} Optimization.

As technologies have shrunk, leakage power consumption has grown exponentially, thus requiring more aggressive power reduction techniques to be used. Similarly, increase in clock frequency have caused dynamic power consumption of the devices to outstrip the capacity of the power networks that supply them, and this

becomes especially acute when high power consumption occurs in very small geometries, as this is a power density issue as well as a power consumption issue.

Several advanced low power techniques have been developed to address these needs. The most commonly adopted techniques today are:

1) Multi-voltage supply (MVS): The operation of different areas of a design at different voltage levels. Only specific areas that require a higher voltage to meet performance targets are connected to the higher voltage supplies. Other portions of the design operate at a lower voltage, allowing for significant power savings. Multi-voltage supply is generally a technique used to reduce dynamic power, but the lower voltage values also cause leakage power to be reduced.

2) Power gating: The complete shut off of supply nets to different areas of a design when they are not needed (also known as MTCMOS or power shutdown). Since the power has been completely removed from these shutdown areas, the power for these areas is reduced essentially to zero. This technique is used to reduce leakage power.

It is very common to see multi-voltage and power gating used together on the same design, whereby different regions operate at different voltages, and one or more of those regions can also be shutdown.

This section formally defines reversible gate, garbage output, delay, hardware complexity and presents popular reversible fault tolerant gates along with their input-output specifications, transistor and quantum equivalent representations.

3.1.1 REVERSIBLE AND FAULT TOLERANT GATES:

An $n \times n$ reversible gate is a data stripe block that uniquely maps between input vector $I_v = (I_0, I_1, \dots, I_{n-1})$ and output vector $O_v = (O_0, O_1, \dots, O_{n-1})$ denoted as $I_v \leftrightarrow O_v$. Two prime requirements for the reversible logic circuit are as follows :

- There should be equal number of inputs and outputs.
- There should be one-to-one correspondence between inputs and outputs for all possible input-output sequences.

A **Fault tolerant gate** is a reversible gate that constantly preserves same parity between input and output vectors. More specifically, an $n \times n$ fault tolerant gate clarify the following property between the input and output vectors [2]:

$$I_0 \oplus I_1 \oplus \dots \oplus I_{n-1} = O_0 \oplus O_1 \oplus \dots \oplus O_{n-1} \quad (1)$$

Parity preserving property of Eq.1 allows to detect a faulty signal from the circuit's primary output. Researchers [2] have showed that the circuit consist of only reversible fault tolerant gates preserves parity and thus able to detect the faulty signal at its primary output.

3.1.2. QUBIT AND QUANTUM COST

The main difference between the *qubits* and conventional bits is that, qubits can form linear combination of states $|0\rangle$ or $|1\rangle$ called superposition, while the basic states $|0\rangle$ or $|1\rangle$ are an orthogonal basis of two-dimensional complex vector .

A superposition can be denoted as, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, which means the probability of particle being measured in states 0 is $|\alpha|^2$, or results 1 with probability $|\beta|^2$, and ofcourse $|\alpha|^2 + |\beta|^2 = 1$. Thus, information stored by a qubit are different when given different α and β . Because of such properties, qubits can perform certain calculations exponentially faster than conventional bits.

This is one of the main motivation behind the quantum computing. Quantum computer demands its underneath circuitry be reversible .The *quantum cost* for all 1×1 and 2×2 reversible gates are considered as 0 and 1, respectively. Hence, quantum cost of a reversible gate or circuit is the total number of 2×2 quantum gate used in that reversible gate or circuit.

3.1.3. DELAY, GARBAGE OUTPUT AND HARDWARE COMPLEXITY:

The *delay* of a circuit is the delay of the critical path. The path with maximum number of gates from any input to any output is the critical path . There may be more than one critical path in a circuit and it is an NP-complete problem to find all the critical paths . So, researchers pick the path which is the most likely candidates for the critical paths .

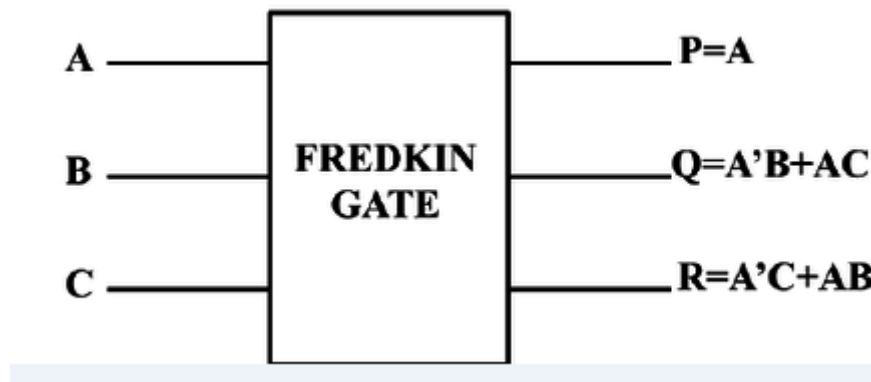
Unused output of a reversible gate (or circuit) is known as *garbage output*, i.e., the output which are needed only to maintain the reversibility are the garbage output. The number of basic operations (Ex-OR, AND, NOT etc.) needed to realize the circuit is referred to as the *hardware complexity* of the circuit.

Actually, a constant complexity is assumed for each basic operation of the circuit, such as, α for Ex-OR, β for AND, γ for NOT etc. Then, total number of operations are calculated in terms of α , β , and γ .

3.1.4. POPULAR REVERSIBLE FAULT TOLERANT GATES

3.1.4.1: FREDKIN GATE:

The **Fredkin gate** (also **CSWAP gate**) is a computational circuit suitable for reversible computing, invented by Ed Fredkin. It is *universal*, which means that any logical or arithmetic operation can be constructed entirely of Fredkin gates. The Fredkin gate is the three-bit gate that swaps the last two bits if the first bit is 1.



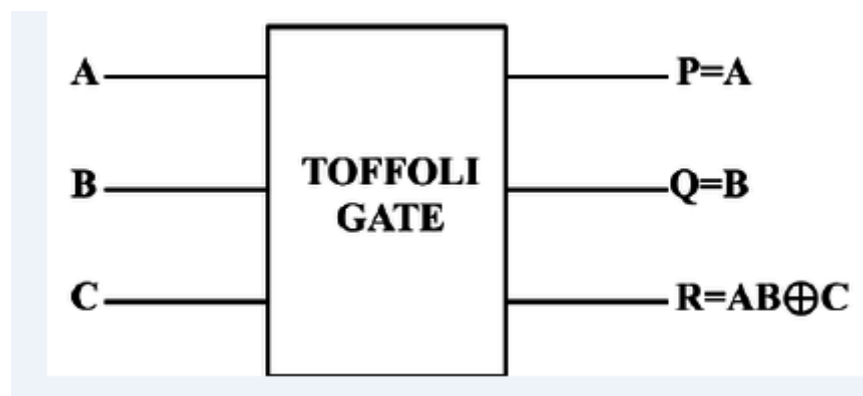
3.1.4.1.1: block diagram of fredkin gate.

The basic Fredkin gate^[1] is a controlled swap gate that maps three inputs (C, I_1, I_2) onto three outputs (C, O_1, O_2) . The C input is mapped directly to the C output. If $C = 0$, no swap is performed; I_1 maps to O_1 , and I_2 maps to O_2 . Otherwise, the two outputs are swapped so that I_1 maps to O_2 , and I_2 maps to O_1 . It is easy to see that this circuit is reversible, i.e., "undoes" itself when run backwards. A generalized $n \times n$ Fredkin gate passes its first $n-2$ inputs unchanged to the corresponding outputs, and swaps its last two outputs if and only if the first $n-2$ inputs are all 1. The Fredkin gate is the reversible three-bit gate that swaps the last two bits if the first bit is 1.

3.1.4.2: TOFFOLI GATES:

In logic circuits, the **Toffoli gate** (also **CCNOT gate**), invented by Tommaso Toffoli, is a universal reversible logic gate, which means that any reversible circuit can be constructed from Toffoli gates.

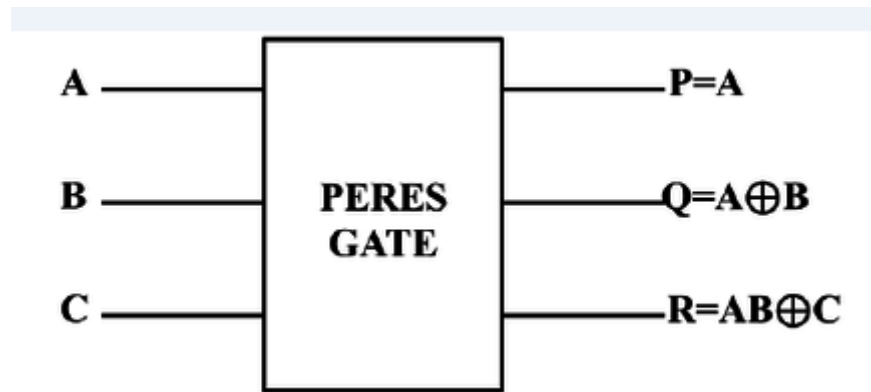
It is also known as the "controlled-controlled-not" gate, which describes its action. It has 3-bit inputs and outputs; if the first two bits are set, it inverts the third bit, otherwise all bits stay the same.



Reversible gates have been studied since the 1960s. The original motivation was that reversible gates dissipate less heat (or, in principle, no heat). In a normal gate, input states are lost, since less information is present in the output than was present at the input. This loss of information loses energy to the surrounding area as heat, because of thermodynamic entropy. Another way to understand this is that charges on a circuit are grounded and thus flow away, taking a small quantity of energy with them when they change state. A reversible gate only moves the states around, and since no information is lost, energy is conserved.

3.1.4.3: PERES GATE :

Peres Gate (PG) is a reversible logic gates. Reversible logic gates are attracting a lot of attention due to their zero power dissipation under ideal conditions. Reversible logic circuits are useful for constructing quantum computers.



Day by day aggressive scaling down of CMOS devices results in several physical limits such as high leakage current, high power density levels, high lithography cost, and limitation of speed in GHz range. It is predicted that these limitations foreshadow the eventual end of scaling trend of traditional CMOS technology.

3.1.4.4: FEYNMAN DOUBLE GATE:

The input vector is I (A, B, C) and the output vector is O (P, Q, R). The outputs are defined by $P=A$, $Q=A\oplus B$, $R=A\oplus C$. Table 1 represents the truth table of Double Feynman gate and figure 6 shows the block diagram of this gate in QCA.

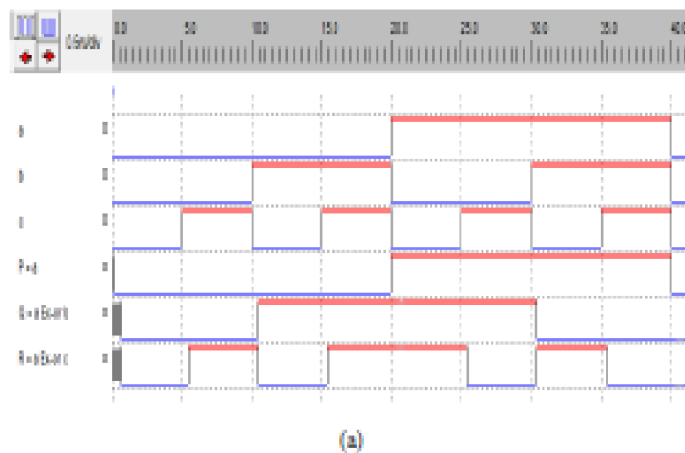


Our proposed circuit was functionally simulated using the QCA Designer [16]. The following parameters are used for a Bi stable Approximation: cell size=18nm,

number of samples=50000, convergence tolerance=0.0000100, radius of effect=65.000000nm, relative permittivity=12.900000.

3.1.5: DECODER:

Decoders are the collection of logic gates fixed up in a specific way such that, for an input combination, all outputs *terms* are low except one. These *terms* are the *minterms*. Thus, when an input combination changes, two outputs will change. Let, there are n inputs, so number of outputs will be 2^n . There are several designs of reversible decoders in the literature. To the best of our knowledge, the designs from [3] is the only reversible design that preserve parity too.



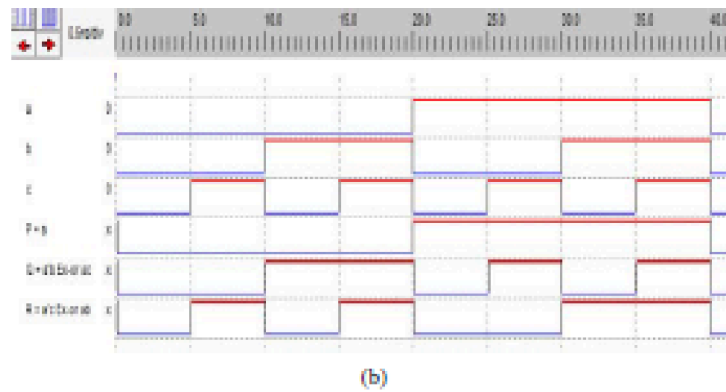
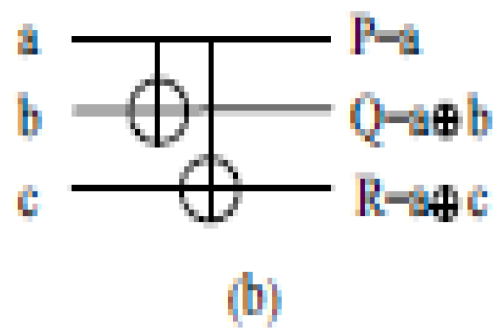
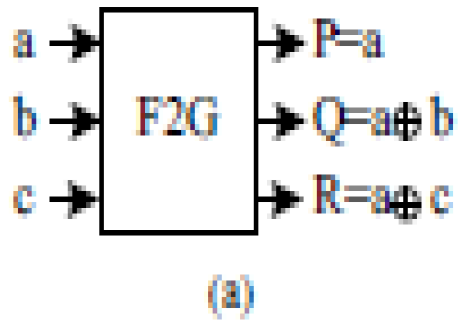


Fig. 3.1.5.1: Simulation of the transistor realization.

3.2 BLOCK DIAGRAM OF FEYNMAN DOUBLE GATE

Input vector (I_v) and output vector (O_v) for 3×3 reversible Feynman double gate ($F2G$) is defined as follows : $I_v = (a, b, c)$ and $O_v = (a, a \oplus b, a \oplus c)$. Block diagram of $F2G$ is shown in Fig. 3.2.1(a). Fig. 3.2.1(b) represent the quantum equivalent realization of $F2G$. From Fig. 3.2.1(b) we find that it is realized with two 2×2 Ex-OR gate, thus its quantum cost is two. According to our design procedure, twelve transistors are required to realize $F2G$ reversibly as shown in Fig. 3.2.1(c). Fig. 3.2.2(a) represents the corresponding timing diagram of $F2G$. Fig.3.2.2 shows the schematic of $F2G$ in microwind and its corresponding layout is shown in fig.3.2.3 and the simulated schematic is shown in fig.3.2.4.



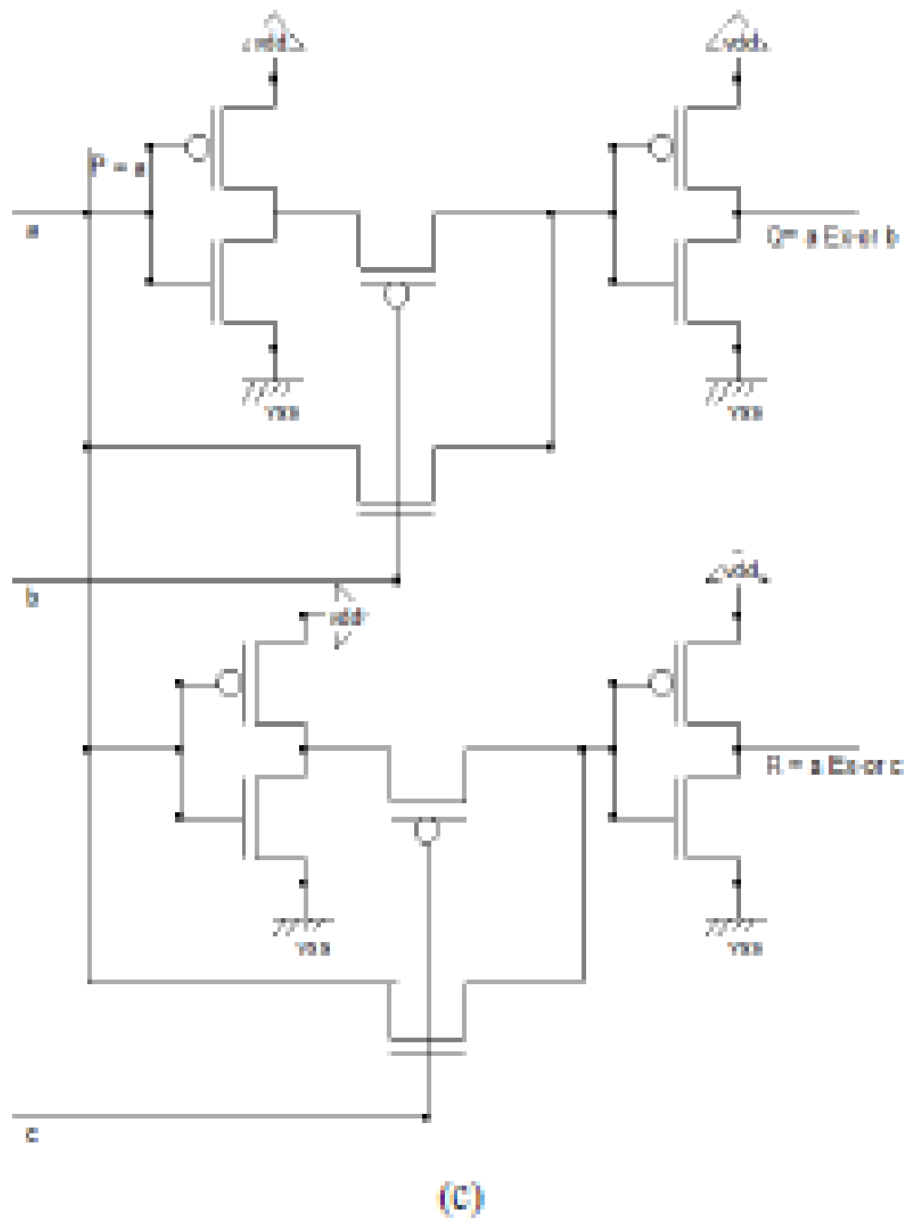


Fig.3.2.1: Reversible Feynman double gate

- (a) Block diagram**
- (b) Quantum equivalent realization**
- (c) Transistor realization**

3.2.1 SCHEMATIC AND LAYOUT OF FEYNMAN DOUBLE GATE

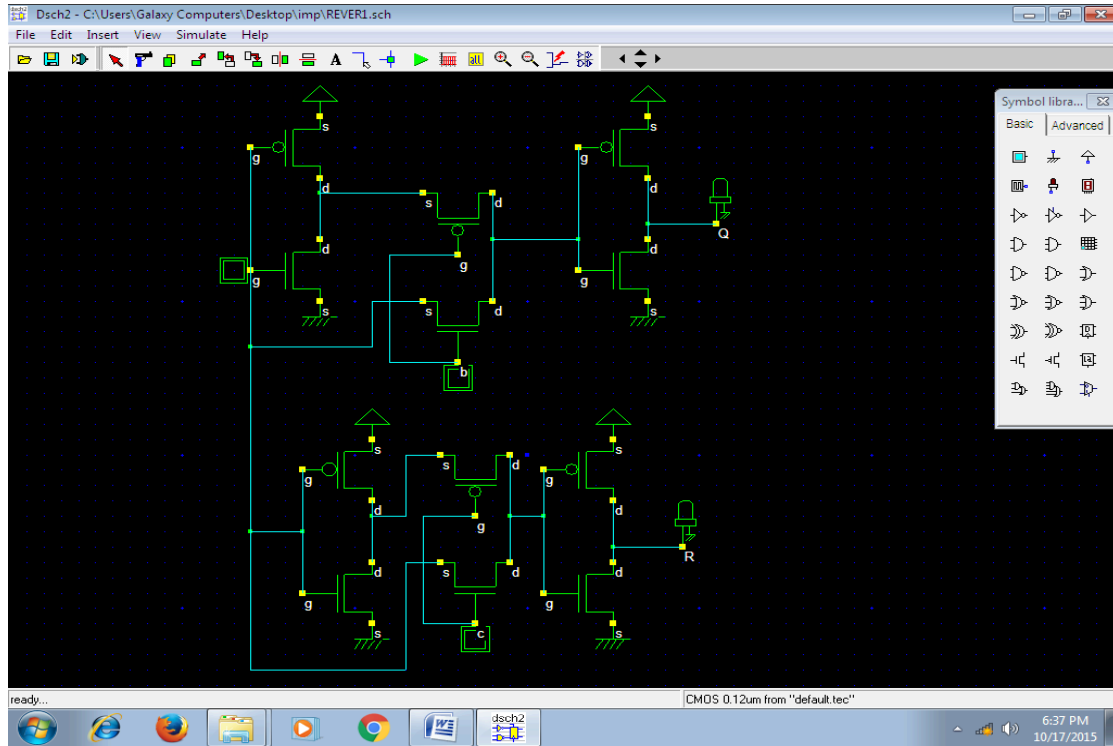


Fig. 3.2.1.1: schematic of Feynman double gate:

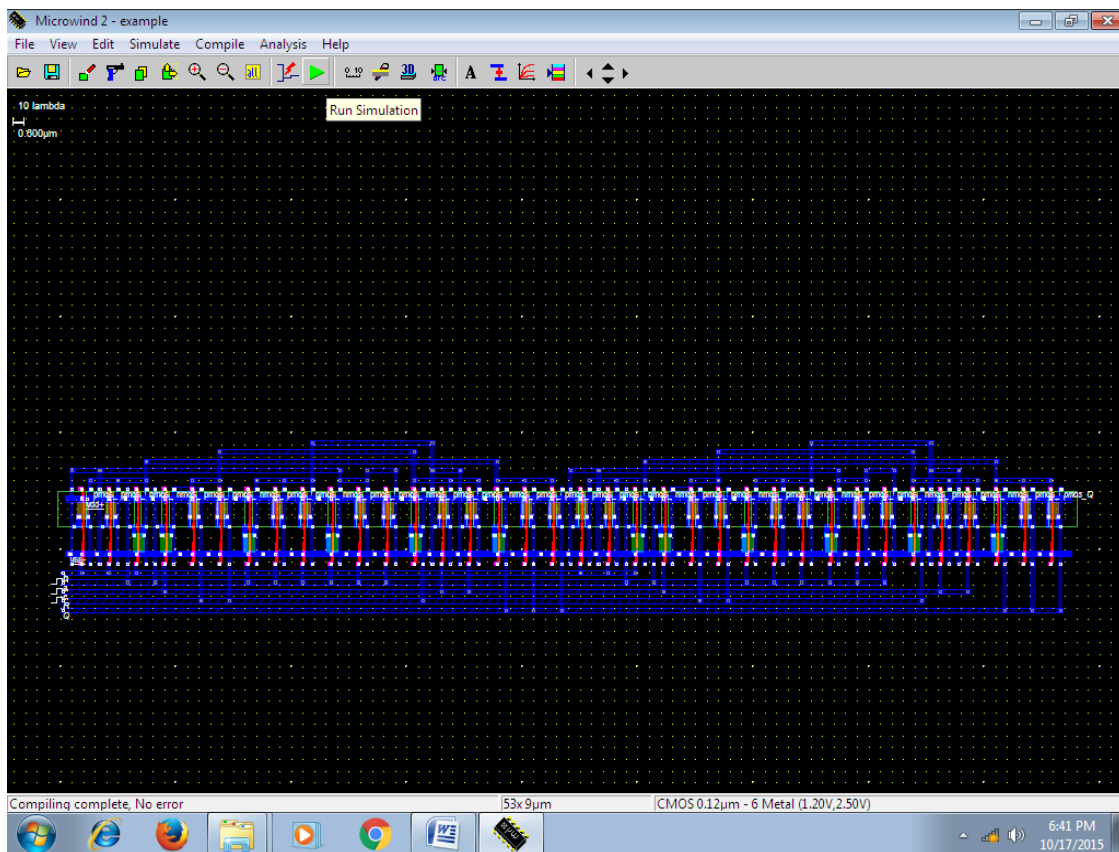


Fig.3.2.1.2: Layout of Feynman double gate

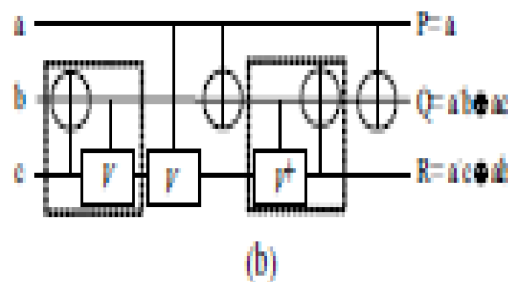
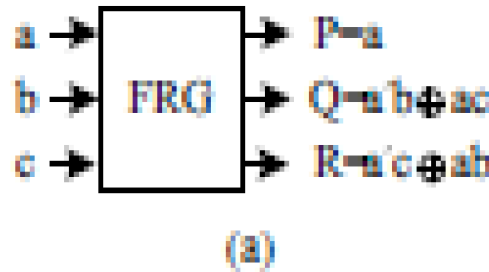
3.3.1 GRAPH AND TIMING DIAGRAM



Fig.3.3.1: Graph & timing diagram of Feynman double gate:

3.3: BLOCK DIAGRAM OF FREDKIN GATE:

The input and output vectors for 3×3 Fredkin gate (*FRG*) are defined as follows : $I_v = (a, b, c)$ and $O_v = (a, a'b \oplus ac, a'c \oplus ab)$. Block diagram of *FRG* is shown in Fig.3.3.1(a). Fig. 3.3.1(b) represents the quantum realization of *FRG*. In Fig. 3.3.1(b), each rectangle is equivalent to a 2×2 quantum primitives, therefore its quantum cost is considered as one . Thus total quantum cost of *FRG* is five. To realize the *FRG*, four transistors are needed as shown in Fig. 3.3.1(c) and its corresponding timing diagram is shown in Fig. 3.3.2(b). Fig.3.3.2 shows the schematic of FRG in microwind and its corresponding layout is shown in fig. 3.3.3 and the simulated schematic is shown in fig.3.3.4.



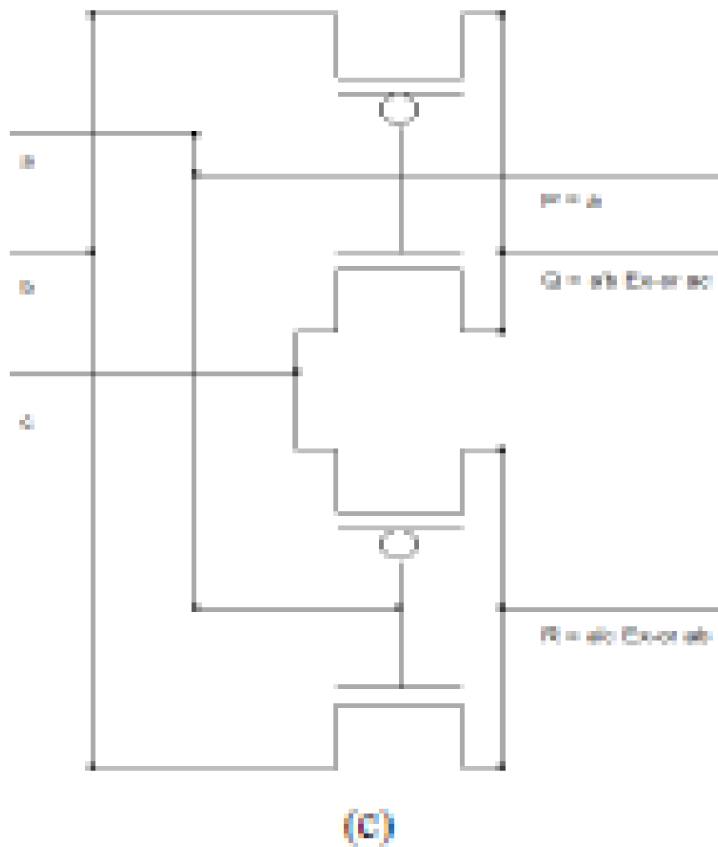


Fig. 3.3.1: Reversible Fredkin gate

- (a) Block diagram
- (b) Quantum equivalent realization
- (c) Transistor realization

3.3.2 SCHEMATIC AND LAYOUT OF FEDKIN GATE

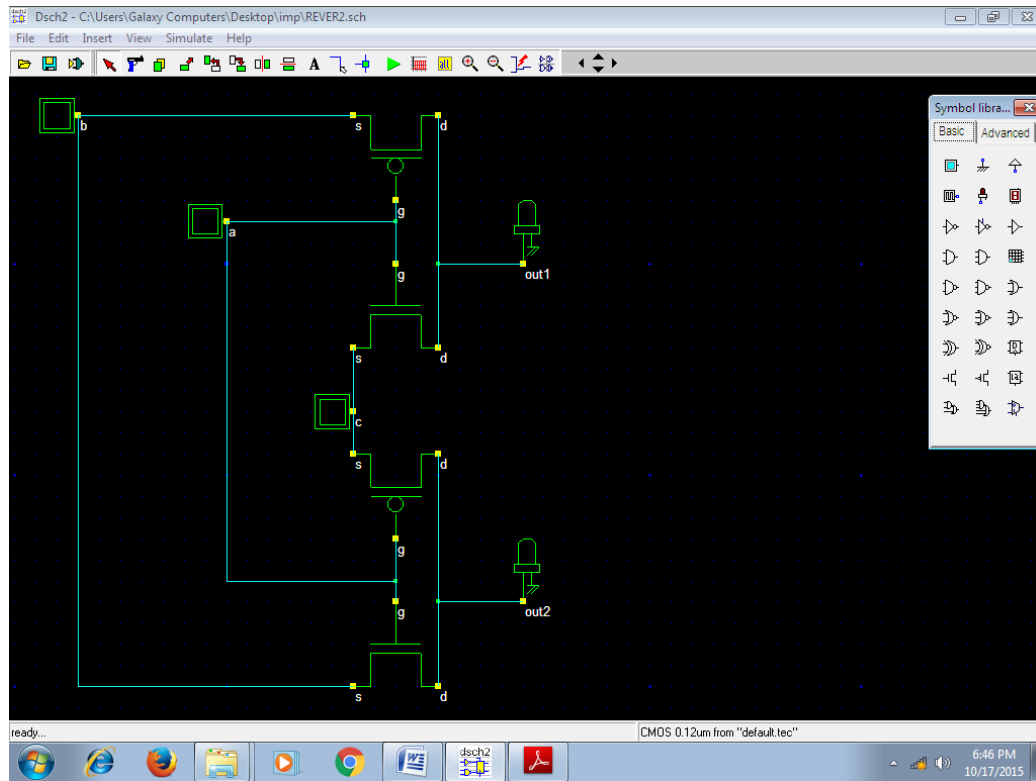


Fig.3.3.2.1: Schematic of Fredkin gate

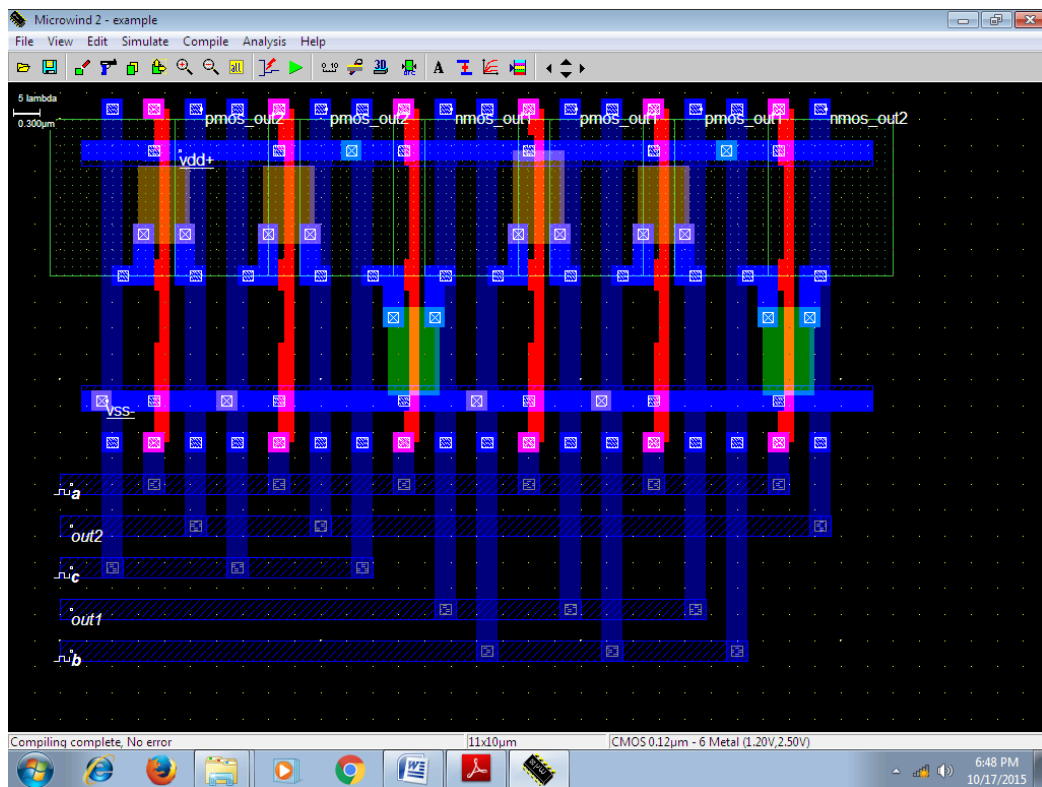


Fig. 3.3.2.2: Layout of Fredkin gate

3.3.3 GRAPH AND TIMING DIAGRAM OF FREDKIN GATE



Fig.3.3.3.1: Graph of Fredkin gate

Reversible Fredkin and Feynman double gate obey the rule of Eq.1. The fault tolerant (parity preserving) property of Fredkin and Feynman double is shown in Table.3.3.1

Input			Output of $F2G$			Output of FRG			Parity
A	B	C	P	Q	R	P	Q	R	
0	0	0	0	0	0	0	0	0	Even
0	0	1	0	0	1	0	0	1	Odd
0	1	0	0	1	0	0	1	0	Odd
0	1	1	0	1	1	0	1	1	Even
1	0	0	1	1	1	1	0	0	Odd
1	0	1	1	1	0	1	1	0	Even
1	1	0	1	0	1	1	0	1	Even
1	1	1	1	0	0	1	1	1	Odd

TABLE 3.3.3.2: Truth table for $F2G$ and FRG

4.1 BLOCK DIAGRAM OF PROPOSED FAULT TOLERANT REVERSIBLE DECODER.

Considering the simplest case, $n=1$, we have a 1-to-2 decoder. Only a $F2G$ can work as 1-to-2 Reversible Fault tolerant Decoder (RFD) as shown in Fig. 4.1.1(a) and its corresponding timing diagram is shown in Fig. 4.1(b). From now on, we denote a reversible fault tolerant decoder as RFD.



Fig. 4.1.1: Proposed 1-to-2 RFD (a) Architecture (b) Simulation.

Fig. 4.1.1(a) represent the architecture of 2-to-4 decoder. Timing diagram of Fig. 4.1.1(a) is shown in Fig. 4.1.1(c). Algorithm 1 presents the design procedure of the proposed n -to- 2^n RFD. Primary input to the algorithm are n control bits. Line 6 of the proposed algorithm assigns the input to the Feynman double gate for the first control bit (S_0), whereas line 9 assigns first two inputs to the Fredkin gates for all the remaining control bits.

Line 10-12 assign third input to the Fredkin gate for $n = 2$. Fig.4.1.1 shows the schematic of the proposed decoder in microwind and its corresponding layout is shown in fig.4.1.2 and the simulated schematic is shown in fig.4.1.3

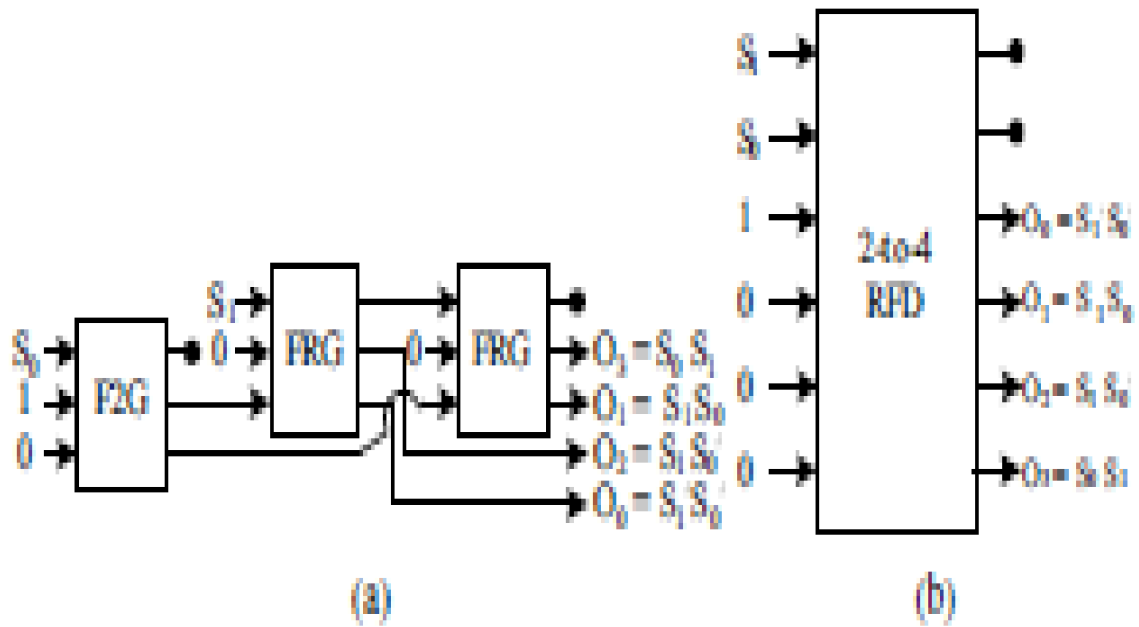


Fig. 4.1.2: (a) Block diagram of the proposed 2-to-4 RFD.

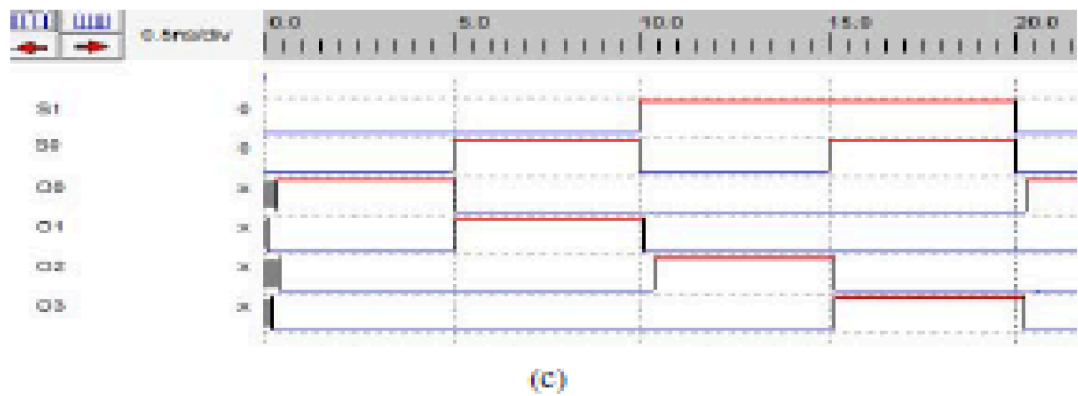


Fig. 4.1.2 (c) Simulation of 2-to-4 RFD

4.2: SCHEMATIC & LAYOUT OF PROPOSED DECODER

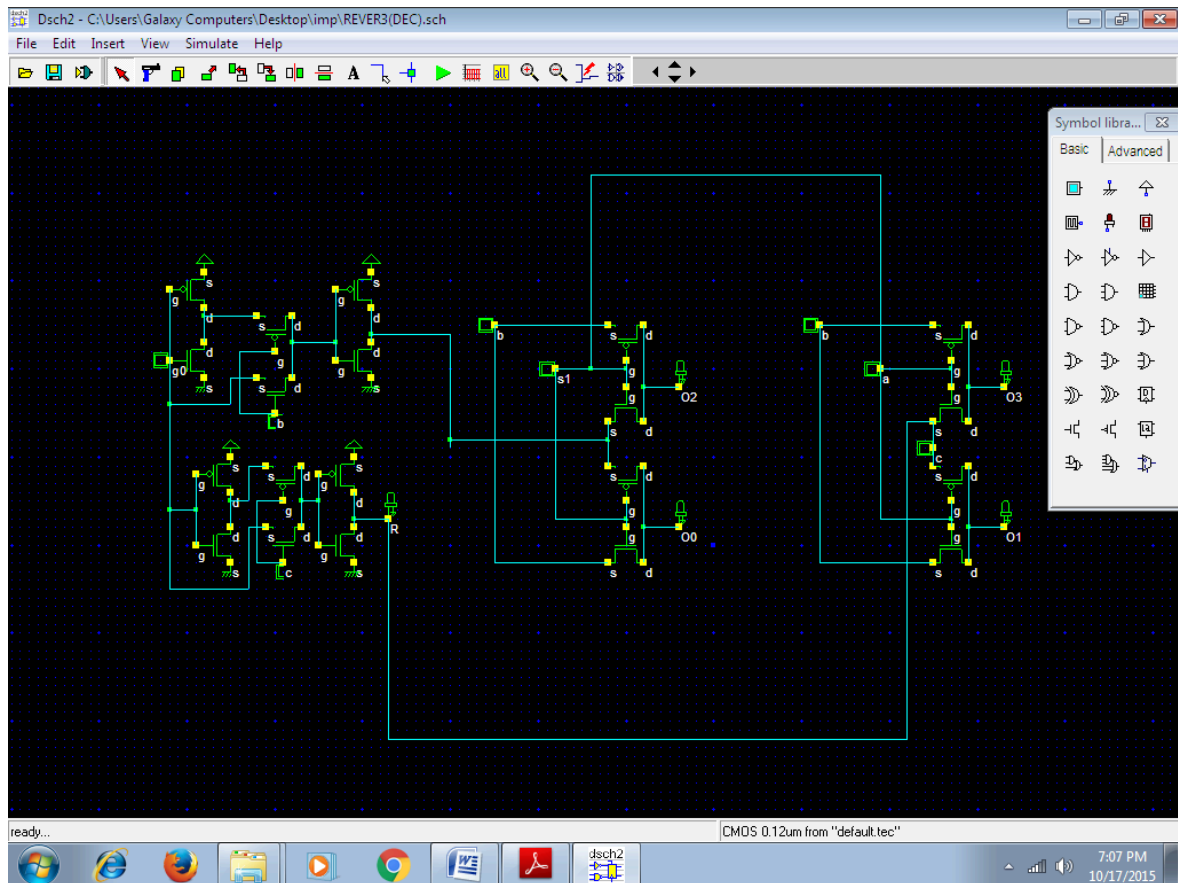
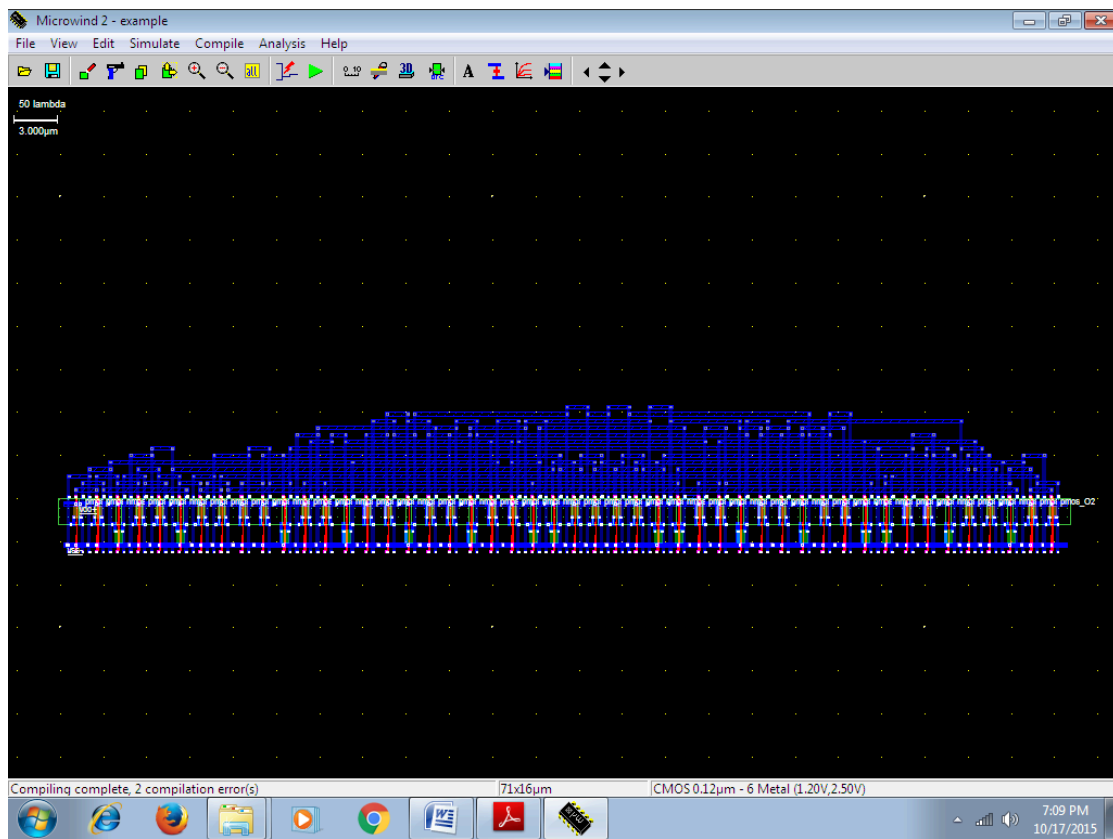


Fig.4.2.1:Schematic & Layout Of Proposed Decoder



4.3 GRAPH AND TIMING DIAGRAM OF PROPOSED DECODER

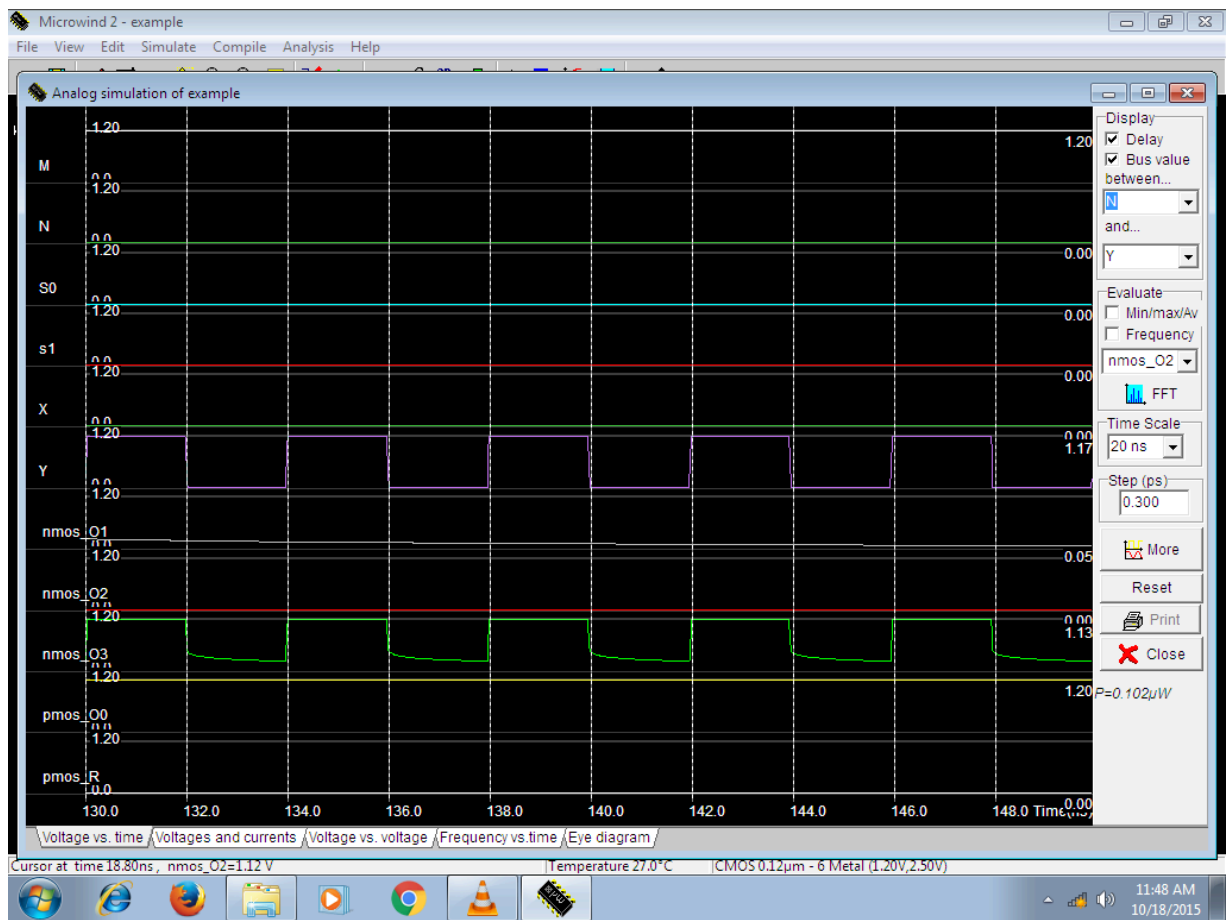


Fig.4.3.1:Graph of proposed decoder

4.4: ALGORITHM OF THE PROPOSED DECODER

Algorithm 1: Algorithm for the proposed reversible fault tolerant n -to- 2^n decoder, $RFD(S, F2G, FRG)$

Input : Data input set $S(S_0, S_1, \dots, S_{n-1})$
 Feynman double gate ($F2G$) and Fredkin gate (FRG)
Output: n -to- 2^n reversible fault tolerant decoder circuit

```

1 begin
2    $i = input$ 
3    $o = output$ 
4   for  $j \leftarrow 0$  to  $n - 1$  do
5     if  $j = 0$  then
6        $S_j \rightarrow first.i.F2G, 1 \rightarrow second.i.F2G, 0 \rightarrow$ 
          $third.i.F2G$ 
7     end if
8     else
9        $S_j \rightarrow first.i.FRg, 0 \rightarrow second.i.FRg$ 
10      if  $j=2$  then
11         $third.o.F2G \rightarrow third.i.FRg$ 
12      end if
13      else
14        call  $RFD(j-1), RFD.o.j \rightarrow third.i.FRg_j$ 
15      end if
16    end if
17  end for
18  return  $FRg.o.3 \ \& \ FRg.o.2 \rightarrow desired \ output$ 
19    remaining  $F2G.o \ \& \ FRg.o \rightarrow garbage$ 
    outputs.
20 end

```

. Each of the proposed circuit is simulated. This simulations also show the functional correctness of the proposed decoders. Table. 4.4.1 shows a comparative study of the proposed fault tolerant decoders with existing fault tolerant one.

	GT	GO	QC	HC	UD
2 to 4 existing circuit [3]	3	2	15	$6\alpha+12\beta+6\gamma$	3
2 to 4 proposed circuit	3	2	12	$6\alpha+8\beta+4\gamma$	2

TABLE 4.4.1: COMPARISON FOR REVERSIBLE DECODERS

5.1 ABOUT MICROWIND

The MICROWIND software allows the designer to simulate and design an integrated circuit at physical description level. Microwind3 unifies schematic entry, pattern based simulator, SPICE extraction of schematic, Verilog extractor, layout compilation, on layout mix-signal circuit simulation, cross sectional & 3D viewer, netlist extraction, BSIM4 tutorial on MOS devices and sign-off correlation to deliver unmatched design performance and designer productivity. The package contains a library of common logic and analog ICs to view and simulate.

Microwind3 includes all the commands for a mask editor as well as, you can gain access to Circuit Simulation by pressing just one single key. The electric extraction of your circuit is automatically performed and the analog simulator produces voltage and current curves immediately. The tool features full editing facilities, various views, and an on-line analog simulator.



Fig 5.1.1 Logo of microwind.

5.2 ABOUT DSCH

The DSCH program is a logic editor and simulator. DSCH is used to validate the architecture of the logic circuit before the microelectronics design is started. DSCH provides a user-friendly environment for hierarchical logic design, and fast simulation with delay analysis, which allows the design and validation of complex logic structures. DSCH also features the symbols, models and assembly support for 8051 and 16F84 controllers. Designers can create logic circuits for interfacing with these controllers and verify software programs using DSCH.

5.2.1 HIGHLIGHTS

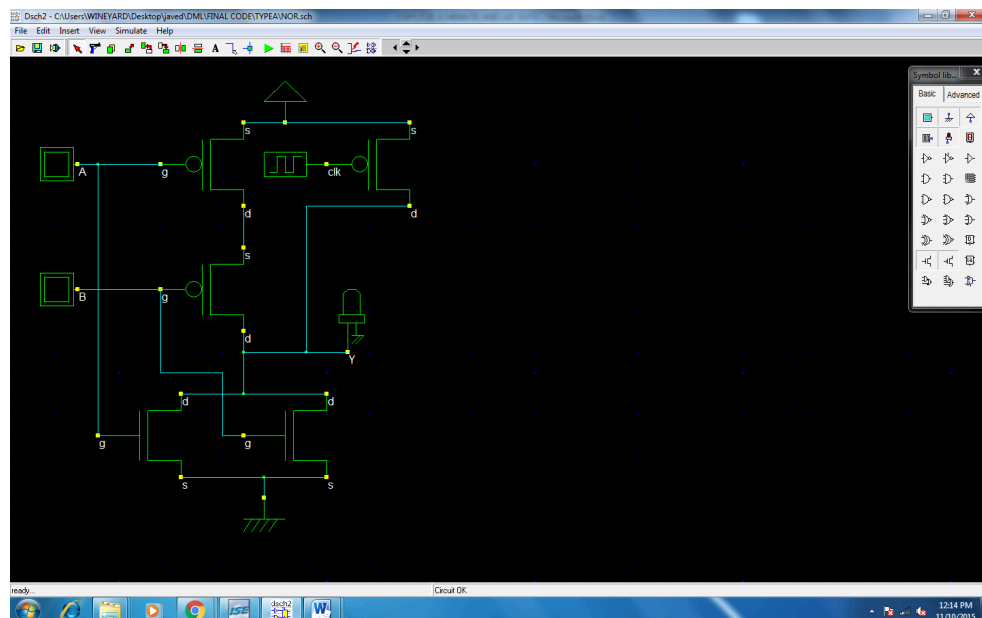
- λ User-friendly environment for rapid design of logic circuits.
- λ Supports hierarchical logic design.
- λ Added a tool on fault analysis at the gate level of digital. Faults: Stuck-1, stuck-at-0.
The technique allows injection of single stuck-at fault at the nodes of the circuit.
- λ Improved interface between DSCH and Winspice.

- λ Handles both conventional pattern-based logic simulation and intuitive on screen mouse-driven simulation.
- λ Built-in extractor which generates a SPICE netlist from the schematic diagram (Compatible with PSPICE™ and WinSpice™).
- λ Generates a VERILOG description of the schematic for layout conversion.
- λ Immediate access to symbol properties (Delay, fanout).
- λ Model and assembly support for 8051 and PIC 16F84 microcontrollers.
- λ Sub-micron, deep-submicron, nanoscale technology support.

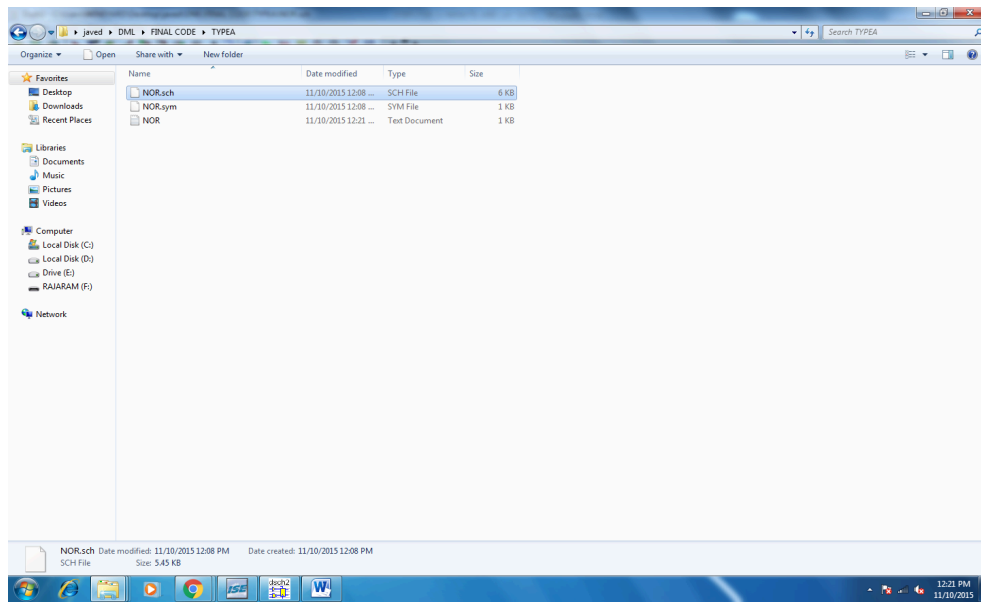
 Supported by huge symbol library

5.3 SOFTWARE STEPS

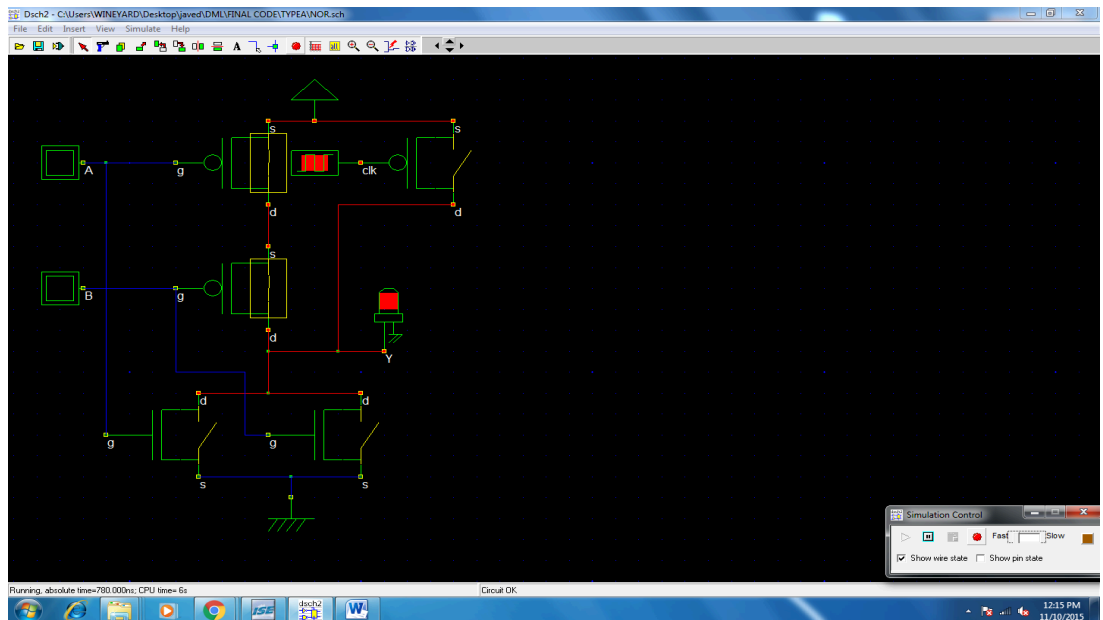
- 1) OPEN Dsch2 Application ICON in Export dsch2 folder
- 2) Take new and draw the schematics –take all basic symbols from symbol library
Ex: (DML NOR GATE)

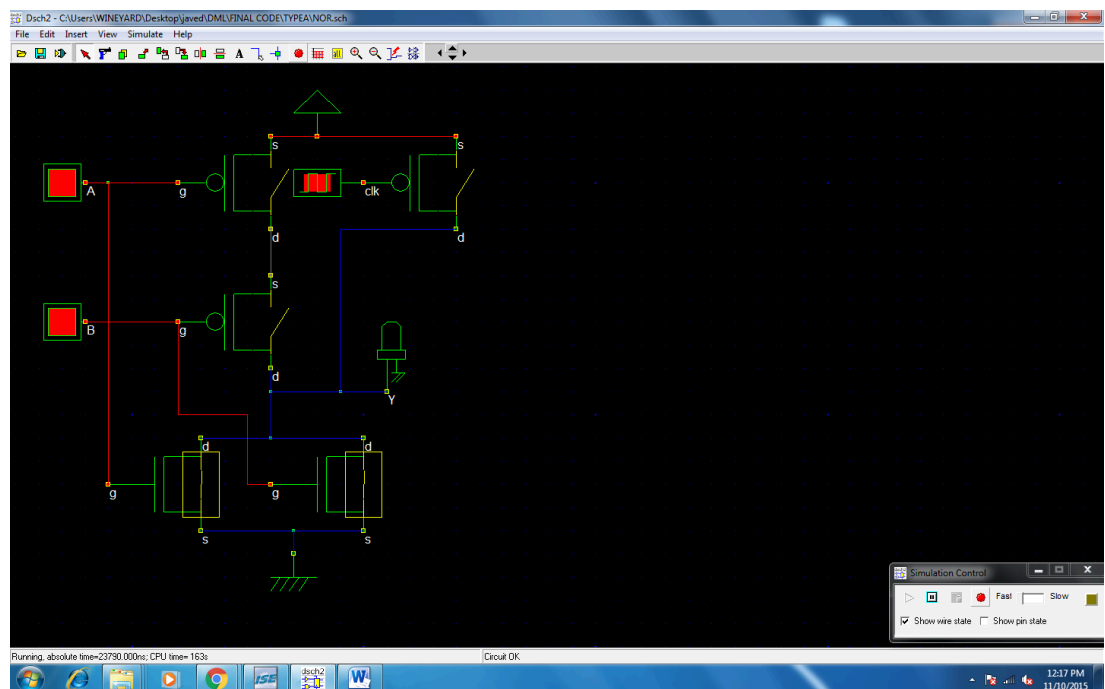
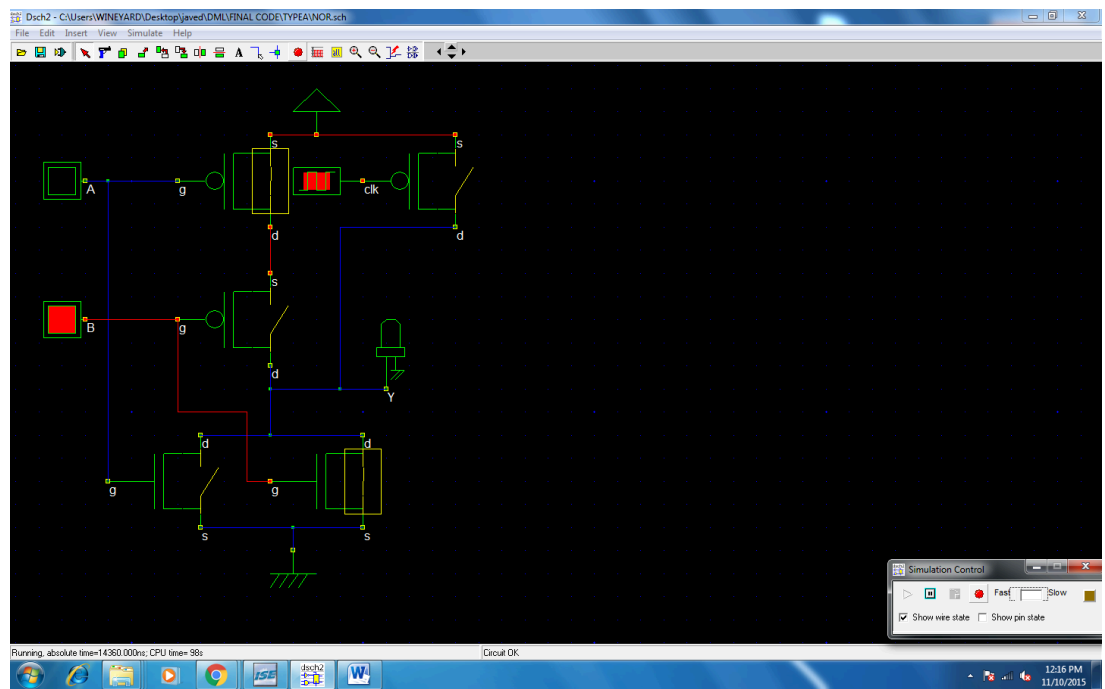


Save schematic with ckt name

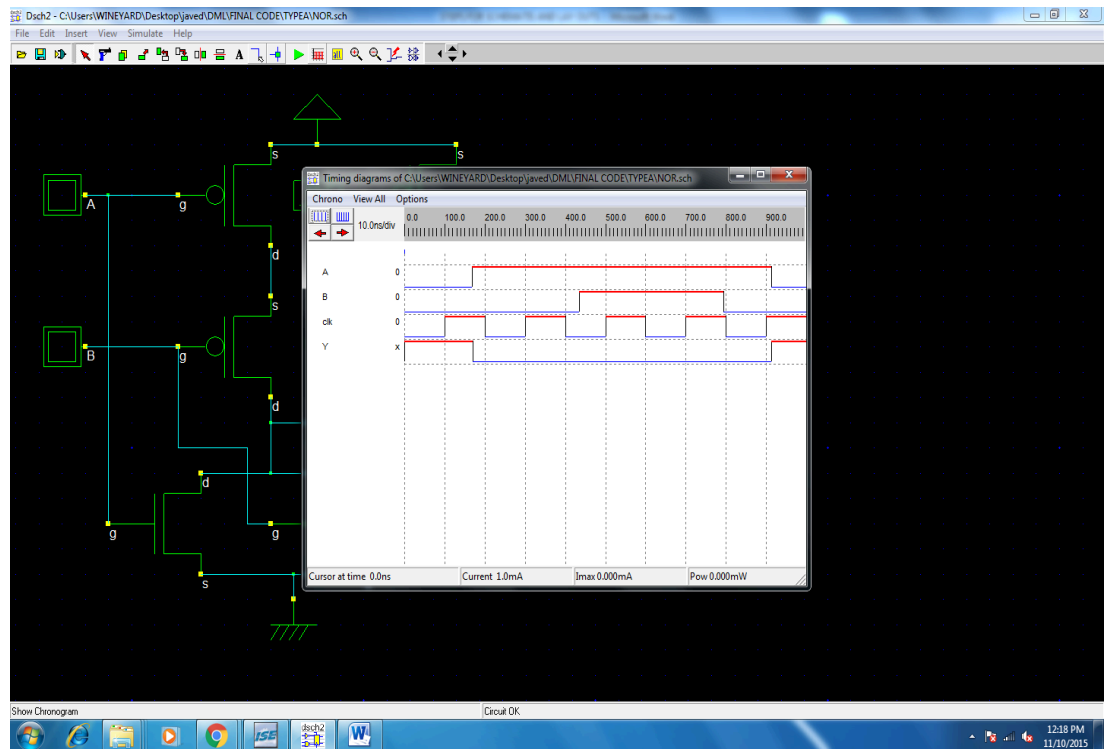


3) Run the simulation

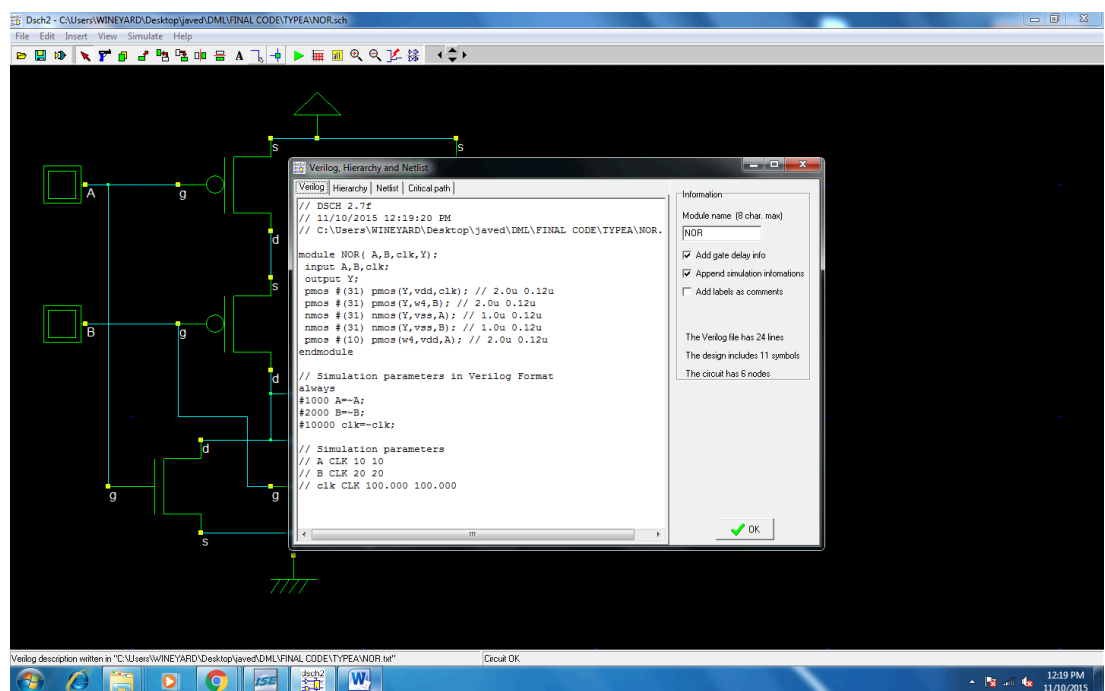
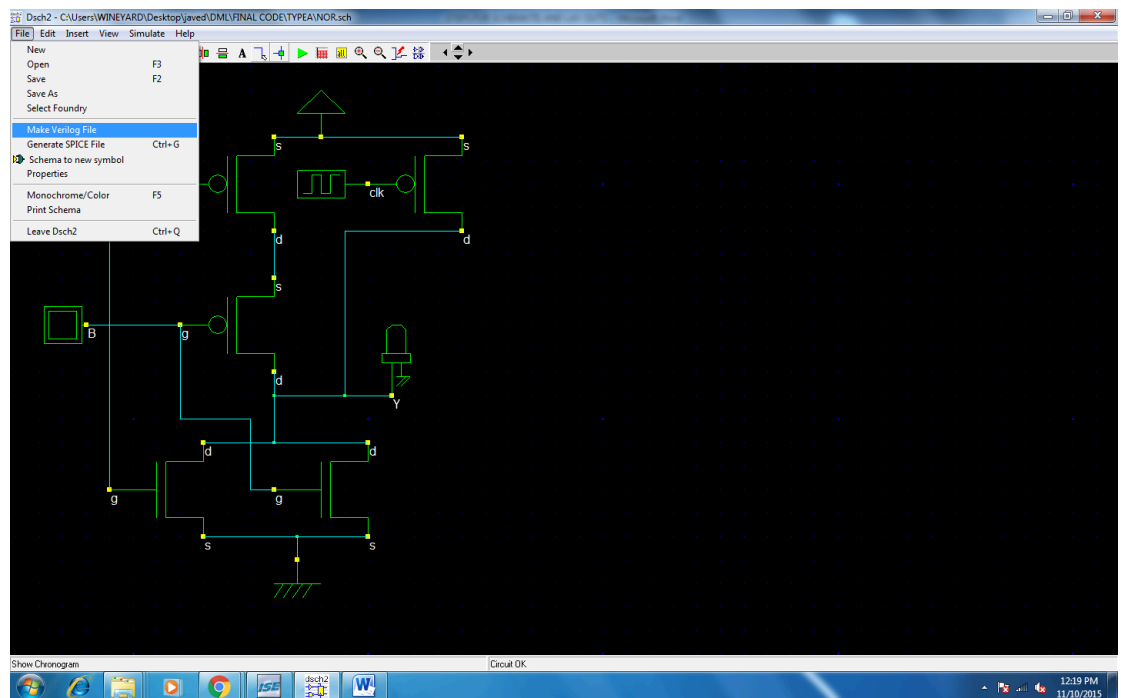




4) observe inputs and outputs on timing diagram



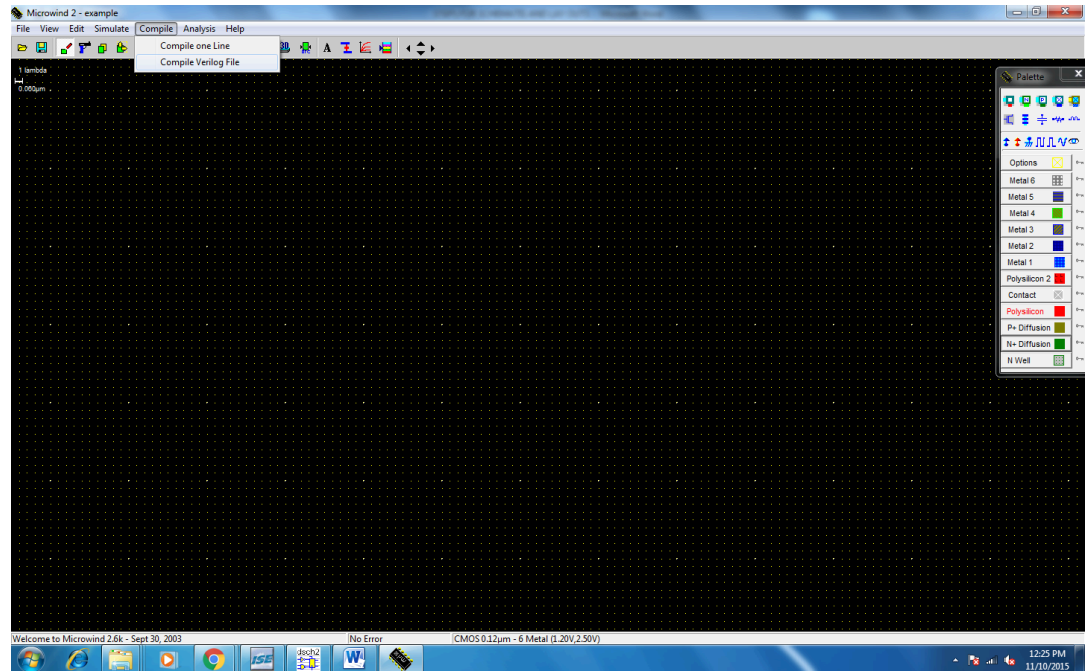
5) generate Verilog file:



click Ok

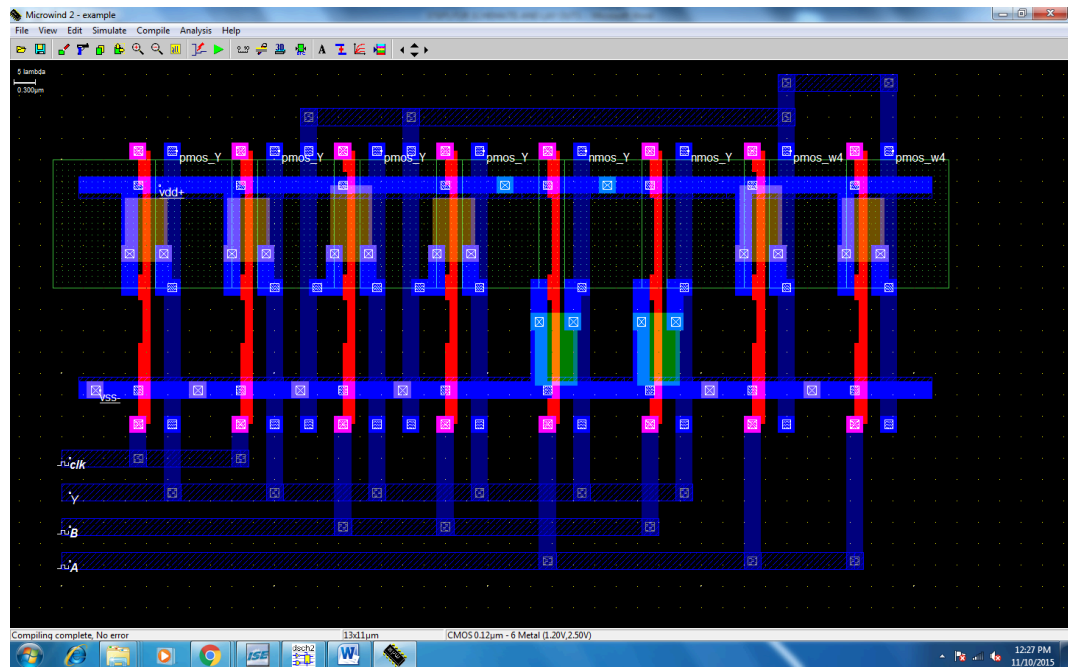
7) open Microwind 2 Application from Export microwind2 folder

8) compile verilog file which is generated from digital schematic

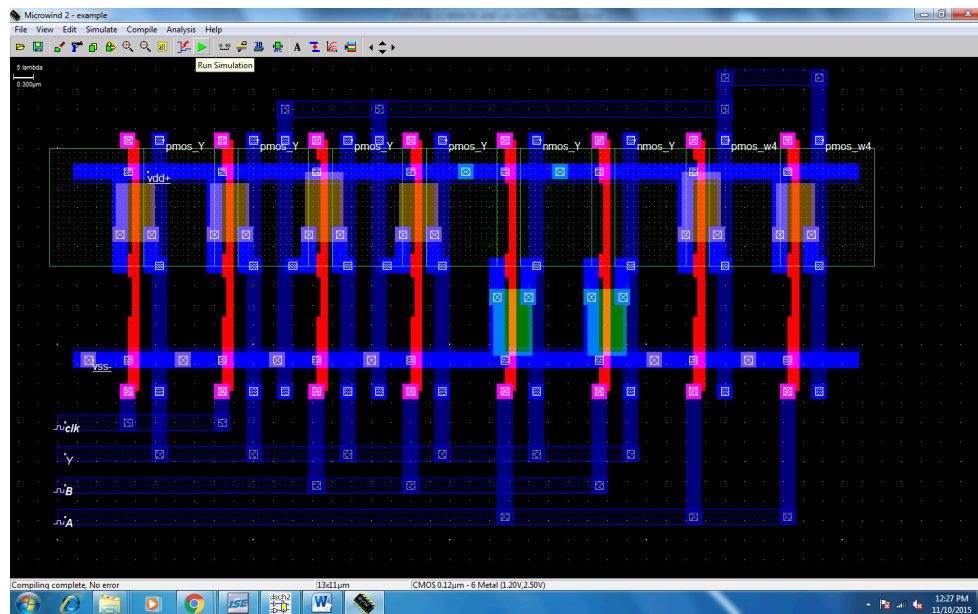


9) compile and back to editor

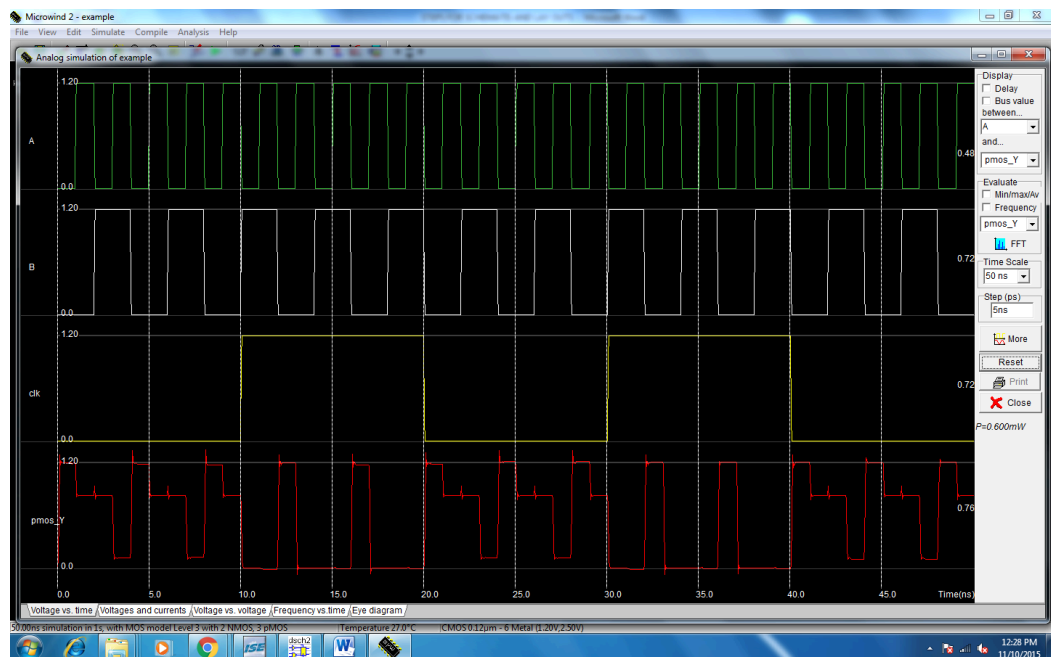
10) observe layouts of ckt



11:run simulation



13) observe simulated output.



6.1 ADVANTAGES:

LESS GARBAGE OUTPUTS.

LESS QUANTUM COST.

LESS HARDWARE COMPLEXITY.

MORE OPTIMIZED IN TERMS OF AREA AND POWER CONSUMPTION.

LESS NUMBER OF GATES.

6.2 DISADVANTAGES:

FEED BACK IS STRICTLY RESTRICTED.

MAXIMUM AND MINIMUM FAN-OUT IS ALWAYS ONE.

LOW AREA AND LESS POWER DISSIPATION.

LOW SPEED AND HIGH DELAY.

6.3 APPLICATIONS

Reversible computing may have applications in computer security and transaction processing, but the main long-term benefit will be felt very well in those areas which require high energy efficiency, speed and performance .it include the area like

1. Low power CMOS.
2. Quantum computer.
3. Nanotechnology
4. Optical computing
5. Design of low power arithmetic and data path for digital signal processing (DSP).
6. Field Programmable Gate Arrays (FPGAs) in CMOS technology for extremely low power, high testability and self-repair

7.1 CONCLUSION

Reversible Computing is an attractive research area. The proposed decoder architectures using the proposed reversible gate offer less hardware complexity and optimized in terms of area and power consumption. Decoders are basic components in digital systems. Furthermore, reversible implementations of these units are necessary for low power VLSI, quantum computers. In this work, we have presented novel design of reversible Decoders which are optimized in terms of garbage outputs, power efficiency number of gates used. We conclude that the choice of reversible gates and the design approach to carefully select a reversible gate for implementing a logic function will significantly impact the garbage outputs, quantum cost and the power efficiency . The design method definitely useful for the construction of future computer and other

computational structures. Alternate optimization methods are under investigation as a future work.

We have presented an approach to the realize the multipurpose binary reversible gates. Such gates can be used in regular circuits realizing Boolean functions. In the same way it is possible to construct multiple-valued reversible gates having similar properties. The proposed asynchronous designs have the applications in digital circuits like a Timer/Counter, building reversible ALU, reversible processor etc. This work forms an important move in building large and complex reversible sequential circuits

8.1 BIBLIOGRAPHY

- [1] B. Parhami, "Fault tolerant reversible circuits," *Proc. of 40th Asimolar Conf. Signals, Systems, and Computers. Pacific Grove, CA*, pp. 1726– 1729, 2006.
- [2] M. Nielsen and I. Chuang, Quantum computation and quantum information. New York, NY, USA: Cambridge University Press, 2000.
- [3] R. Landauer, "Irreversibility and Heat Generation in the Computational Process", IBM Journal of Research and Development, vol. 5, pp. 183-191, 1961.
- [4]C.H. Bennett, "Notes on the History of Reversible Computation", IBM Journal of Research and Development, vol. 32, pp. 16-23, 1998.

[5] Prashant .R.Yelekar, Prof. Sujata S. Chiwande”Introduction to Reversible Logic Gates & its Application” 2nd National Conference on Information and Communication Technology (NCICT) 2011 Proceedings published in International Journal of Computer Applications® (IJCA)

[6] Raghava Garipelly, P.Madhu Kiran, A.Santhosh Kumar,” A review on reversible logic gates and their implementations, International Journal of Emerging Technology and Advanced Engineering ,ISSN 2250-2459, ISO 9001:2008 Certified Journal, Volume 3, Issue 3, March 2013